EF234504 — Grafika Komputer

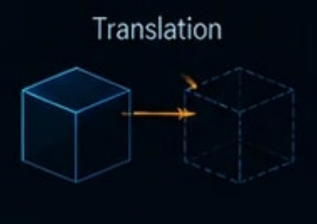
Pertemuan 3

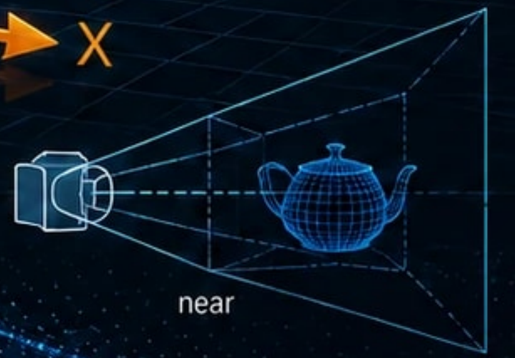
Transformation & Coordinate System

Lab. Grafika, Interaksi, Gim dan Analitik (GIGA)
Departemen Teknik Informatika

$$M = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$









TOPIK PEMBAHASAN

- Coordinate System
- Local Coordinate
- World Coordinate
- View, Clip, NDC, Screen sebagai pipeline overview
- Translation
- Scaling
- Rotation
- Degree dan Radian
- Matrix
- Homogeneous Coordinate
- Translation Matrix
- Rotation Matrix
- Scaling Matrix
- Matrix Multiplication
- Transform Composition
- Transform Order
- Model Matrix
- Uniform Matrix pada WebGL
- **Praktikum:** Interactive Transformation

Capaian Pembelajaran

Mahasiswa mampu:

1. menjelaskan coordinate space pada graphics pipeline
2. membedakan local dan world coordinate
3. menerapkan translation, rotation, dan scaling
4. menjelaskan transformasi menggunakan matrix
5. menjelaskan homogeneous coordinate
6. membentuk Model Matrix
7. menggabungkan transformasi melalui matrix multiplication
8. menjelaskan pengaruh urutan transformasi
9. mengirim transform matrix ke shader melalui uniform
10. membuat objek WebGL yang dapat ditransformasikan secara interaktif

Posisi Materi

PERTEMUAN 2

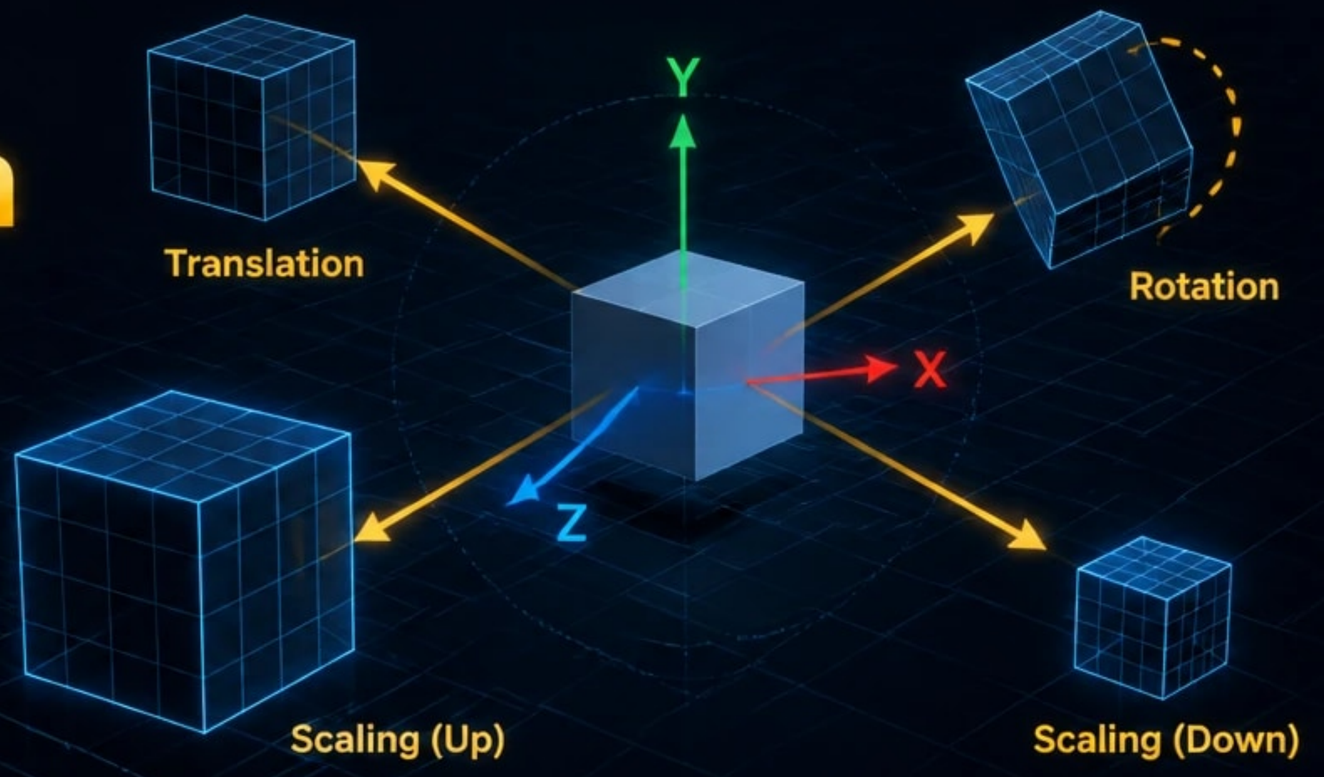
- Vertex Data
- Vertex Shader
- Primitive
- Fragment Shader

PERTEMUAN 3 MENAMBAHKAN

- Vertex
- Transformation Matrix
- Transformed Vertex

KITA **TIDAK MENGUBAH** GEOMETRY ASLI DI BUFFER.

Mengapa Transformation Dibutuhkan?



Object grafika perlu:

- dipindahkan
- diputar
- diperbesar
- diperkecil
- ditempatkan dalam *scene*

Contoh:

Character berjalan → **Translation**



Mobil berbelok → **Rotation**



Object membesar → **Scaling**



Coordinate System

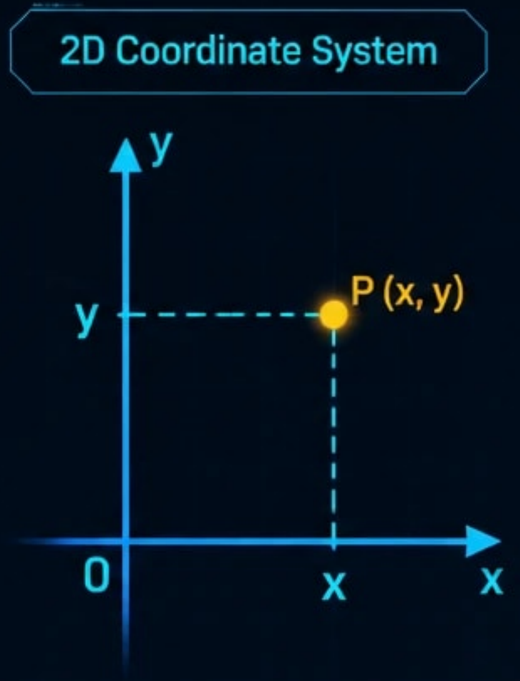
Coordinate system adalah **sistem referensi** untuk menyatakan posisi.

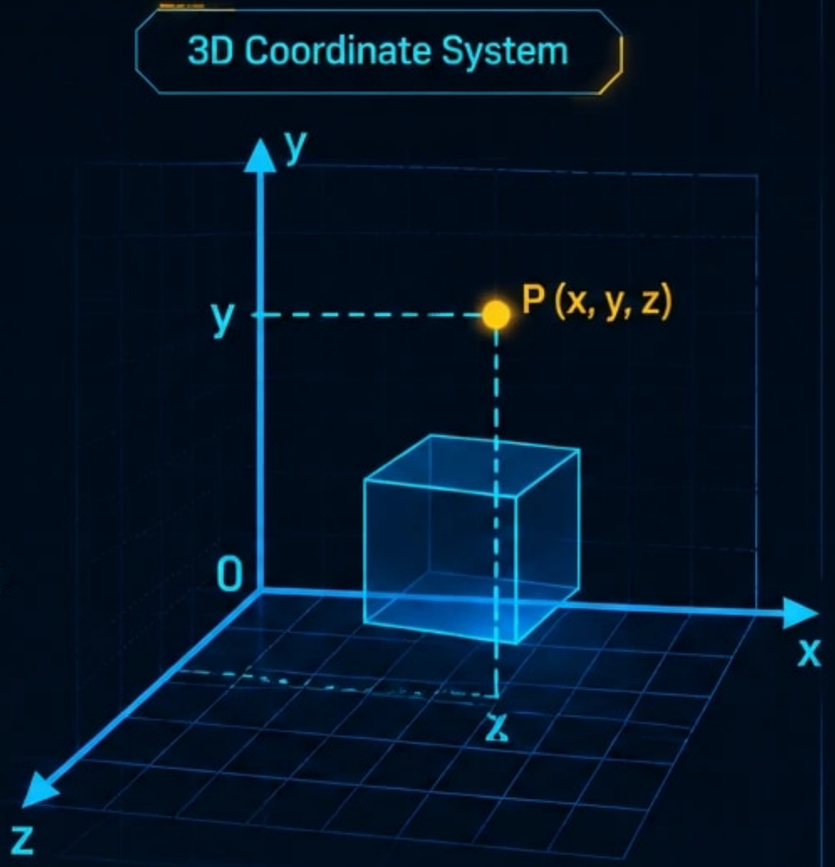
2D:

$P = (x, y)$

3D:

$P = (x, y, z)$





Nilai koordinat sebuah titik bergantung pada **coordinate space**.

Grafika Komputer

Satu Vertex, Banyak Coordinate Space

ALUR TRANSFORMASI VERTEX

- Local
- World
- View
- Clip
- NDC
- Screen

Local (Object Space)

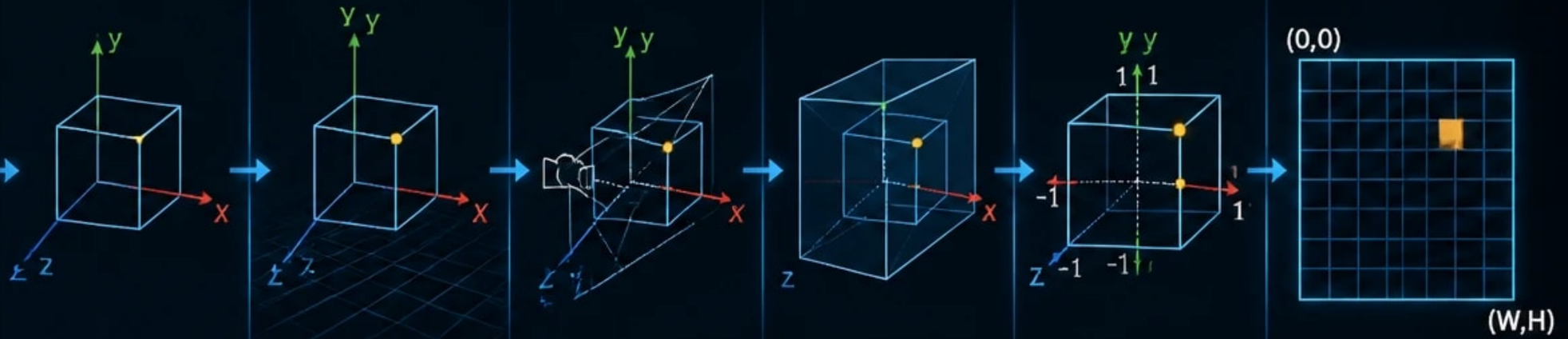
World (World Space)

View (View Space)

Clip (Clip Space)

NDC (Normalized Device Coordinates)

Screen (Screen Space)



PERTEMUAN 3 FOKUS PADA:
Local → World

PERTEMUAN 4 FOKUS PADA:
View dan Projection

Local Coordinate

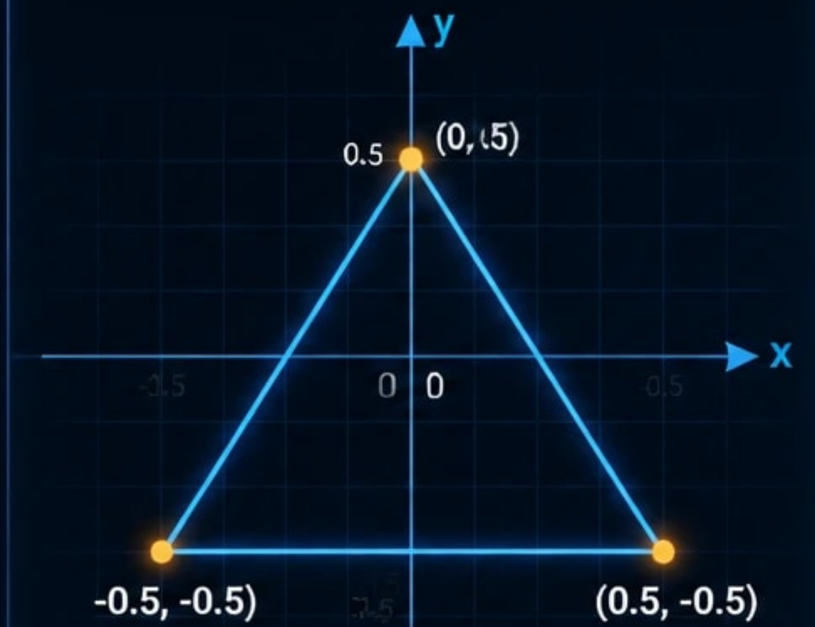
Local coordinate adalah **posisi relatif terhadap origin object.**

Geometry dasar didefinisikan di **local space.**

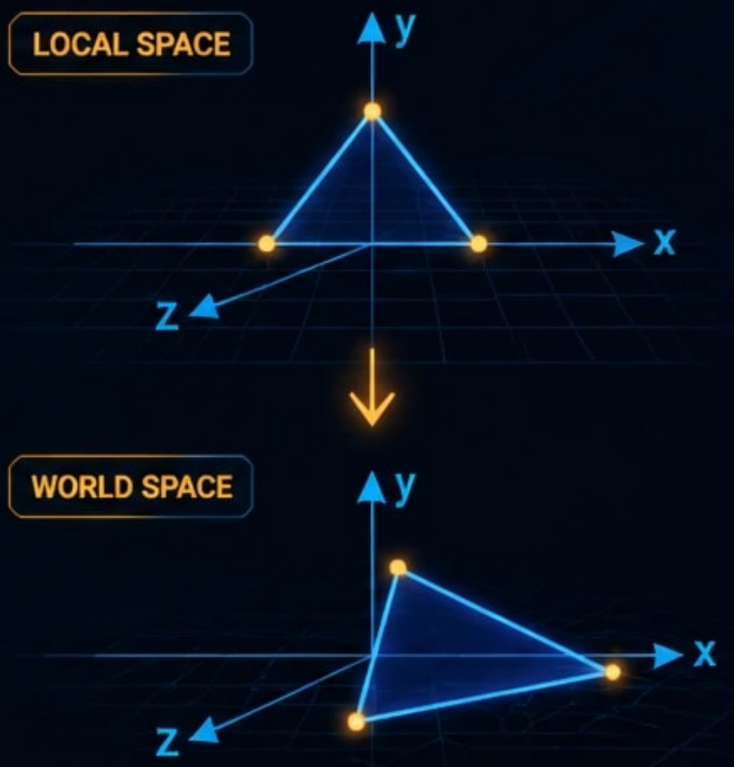
KONSEP UTAMA

- Origin selalu di pusat object
- Sutuan biasanya dinormalisasi
- Independen dari posisi di dunia
- Memudahkan transformasi

CONTOH: SEGITIGA DI LOCAL SPACE



LOCAL VS WORLD SPACE



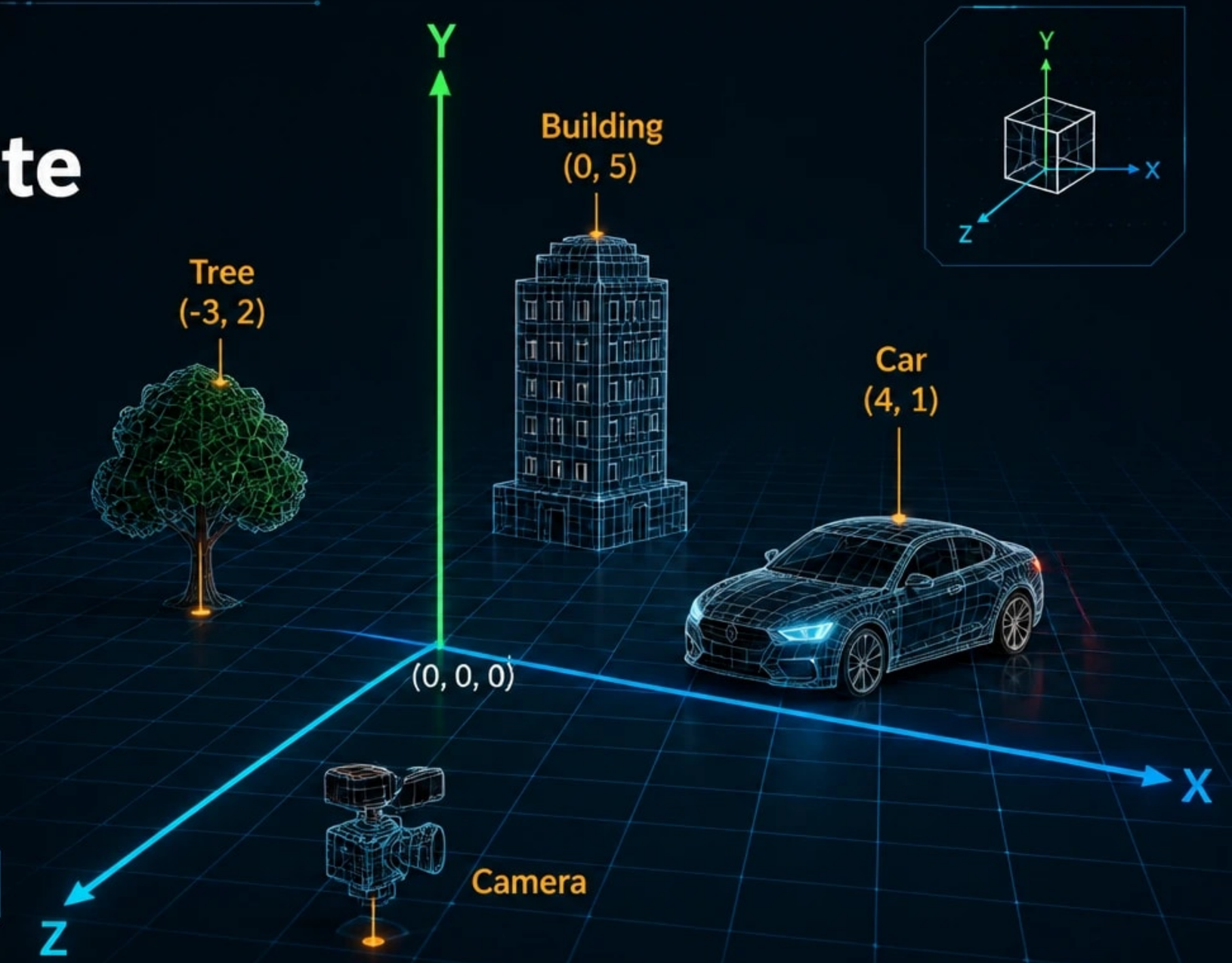
Local coordinate mendefinisikan bentuk object **sebelum ditransformasi** ke **world space.**

World Coordinate

World coordinate adalah coordinate system global sebuah scene.

Car	(4, 1)
Tree	(-3, 2)
Building	(0, 5)
Camera	

Semua object berbagi **world space.**



Apa yang salah dari gambar ilustrasi ini?

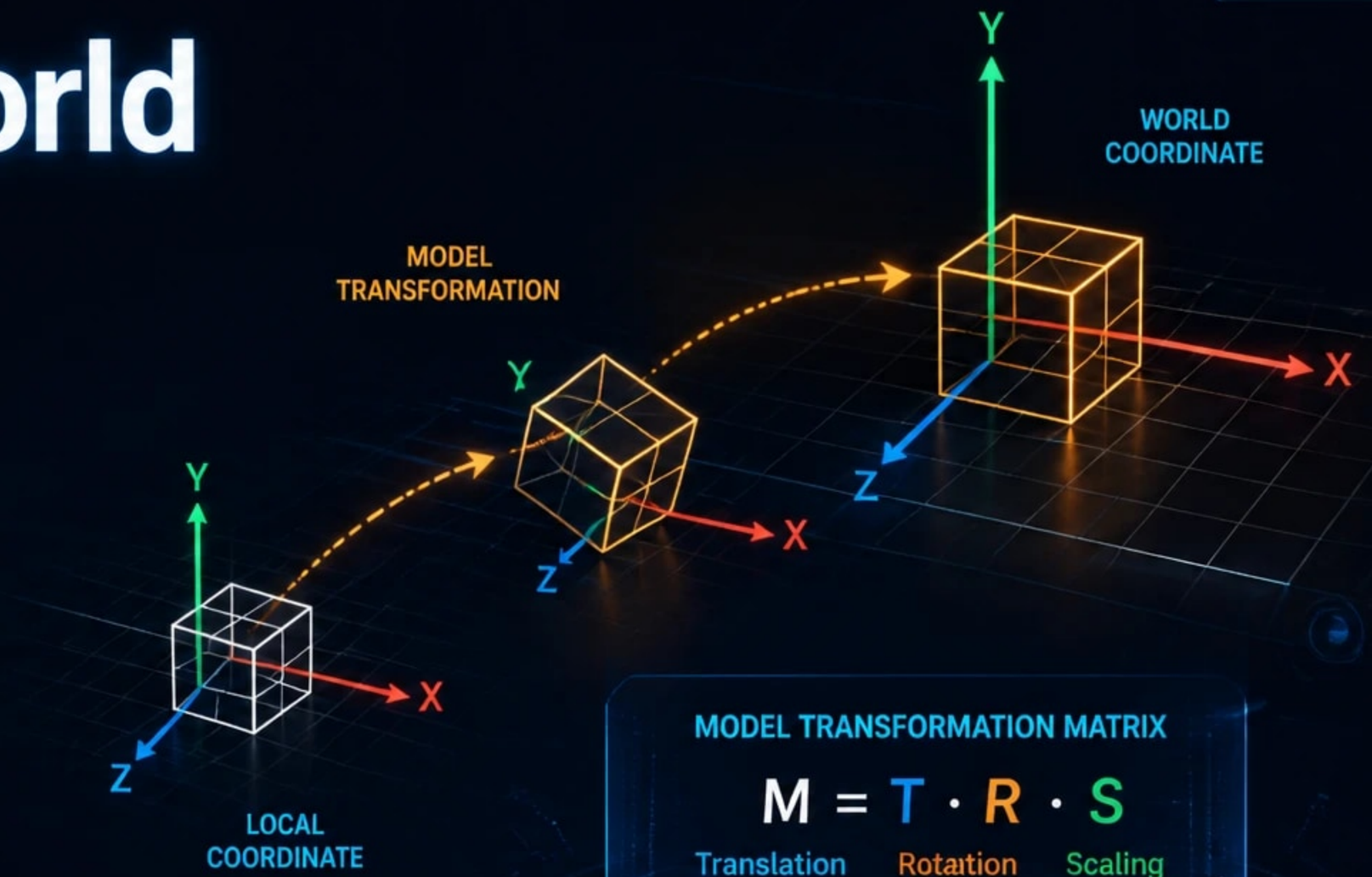
Local ke World

TRANSFORMASI:

- Local Coordinate
- ↓
- Model Transformation
- ↓
- World Coordinate

MODEL TRANSFORMATION DIBENTUK DARI:

Translation + Rotation + Scaling



MODEL TRANSFORMATION MATRIX

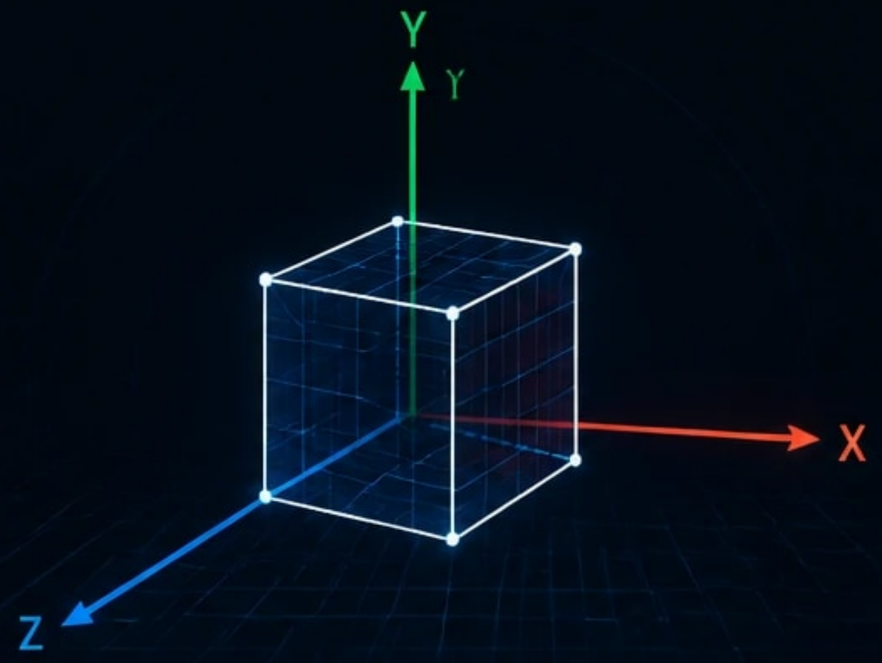
$$M = T \cdot R \cdot S$$

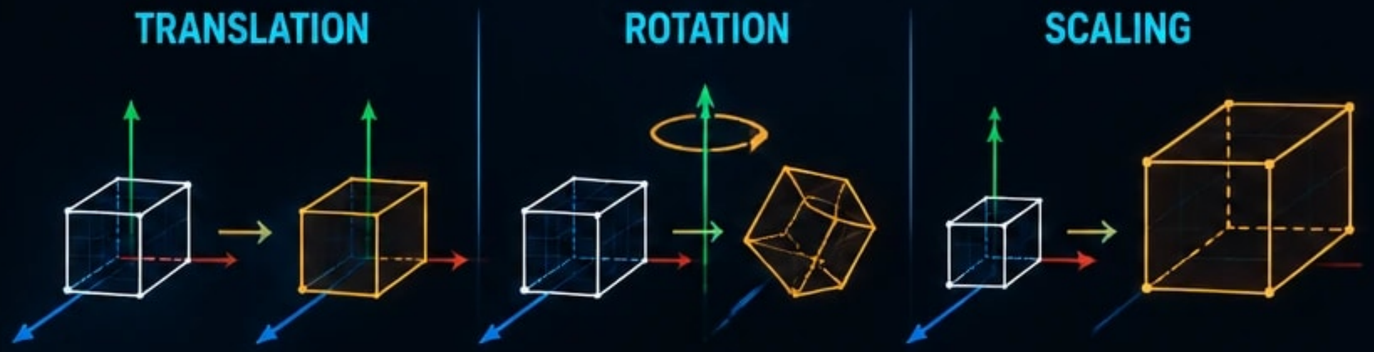
Translation Rotation Scaling

TRANSFORMASI DASAR

TRANSFORMASI		MENGUBAH
TRANSLATION	→	POSISI
ROTATION	→	ORIENTASI
SCALING	→	UKURAN

KETIGANYA DITERAPKAN PADA **VERTEX OBJECT.**





Mengubah posisi object tanpa mengubah orientasi dan ukuran.

Mengubah orientasi object mengelilingi suatu sumbu tanpa mengubah posisi dan ukuran.

Mengubah ukuran object terhadap suatu titik pusat tanpa mengubah posisi dan orientasi.

TRANSLATION

Dalam 2D:

$$x' = x + t_x$$
$$y' = y + t_y$$

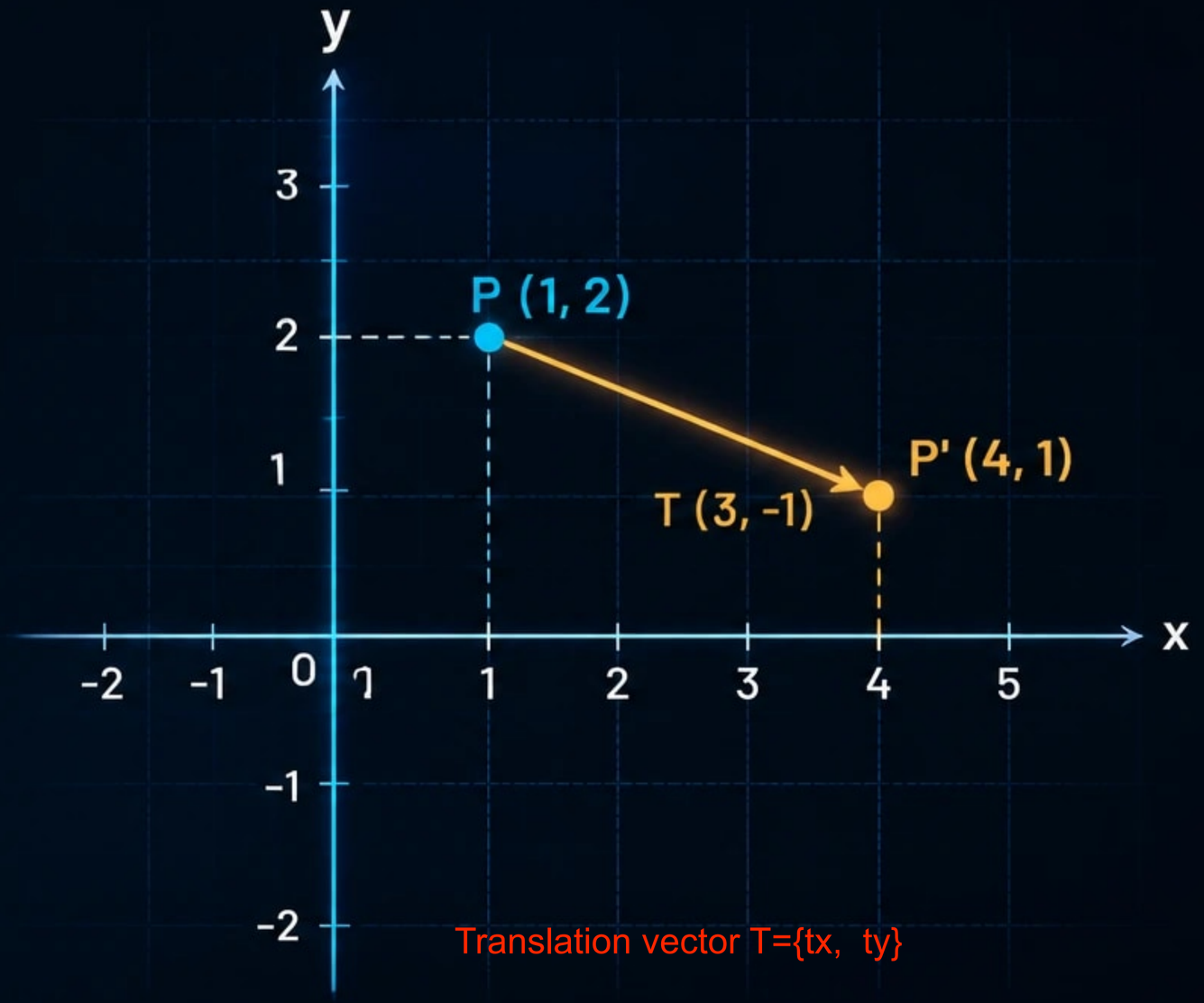
Contoh:

- P = (1, 2)
- T = (3, -1)

- P' = (4, 1)



Bentuk object tidak berubah.



Translation vector T={tx, ty}

Scaling

Dalam 2D:

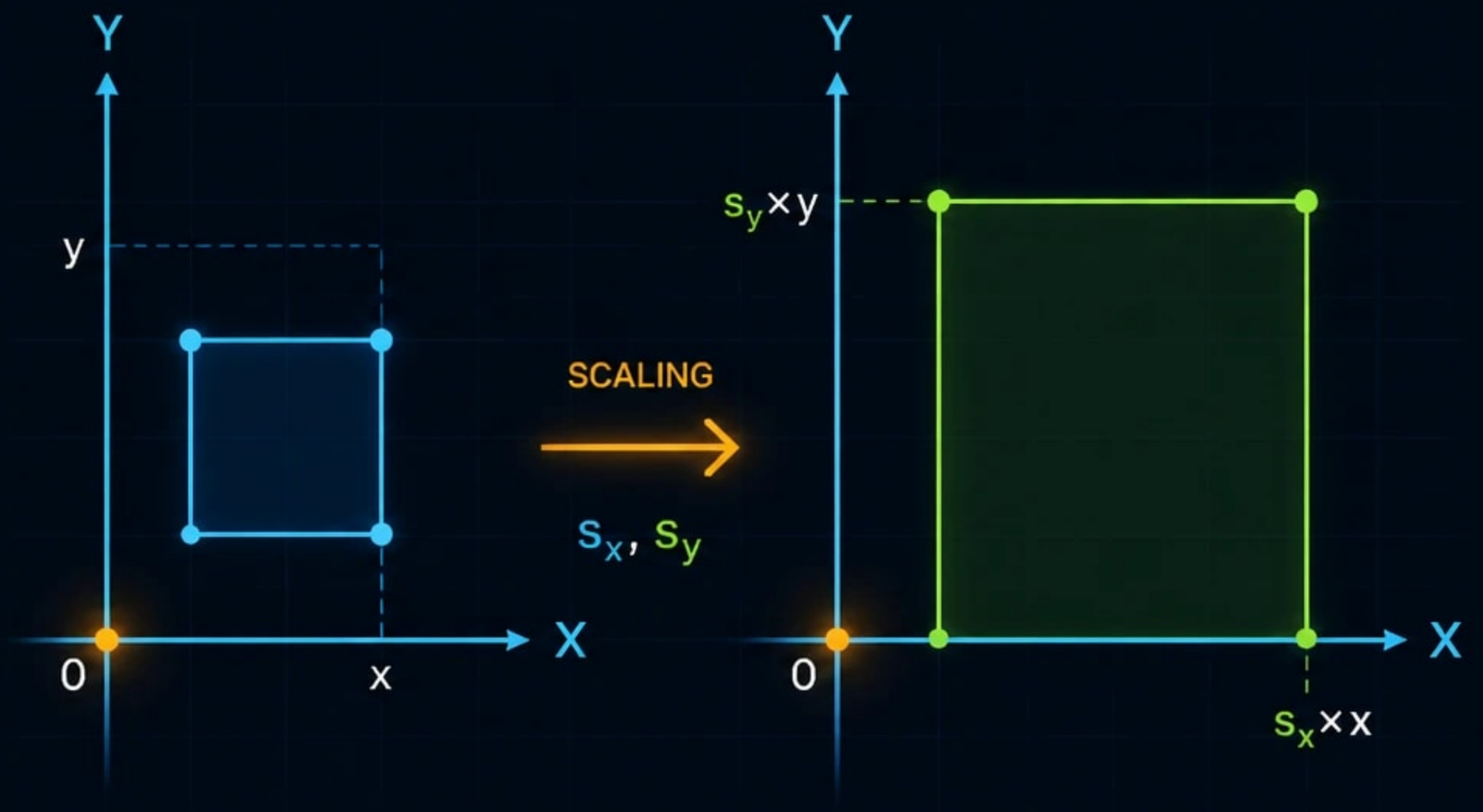
$$x' = s_x \times x$$
$$y' = s_y \times y$$

Dengan:

s_x = scale X

s_y = scale Y

Scaling terjadi relatif terhadap origin/pivot.



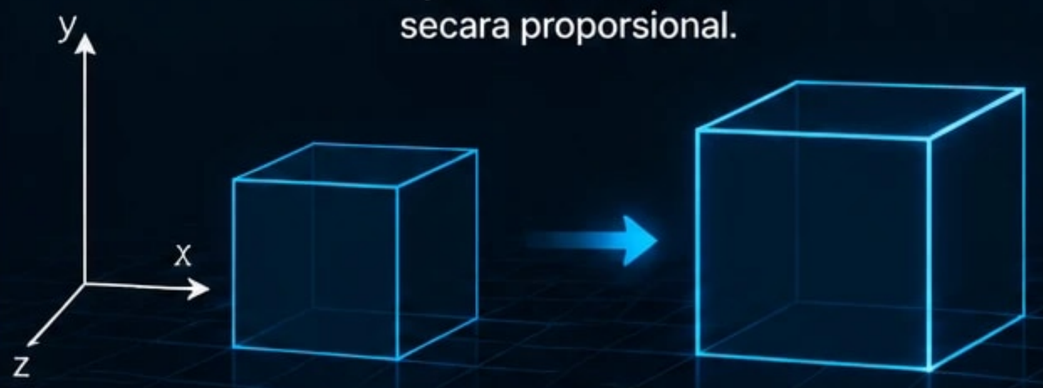
Skala s_x dan s_y mengubah ukuran objek dari origin (0,0).

Uniform vs Non-Uniform Scaling

UNIFORM SCALING

$$s_x = s_y$$

Object berubah ukuran secara proporsional.

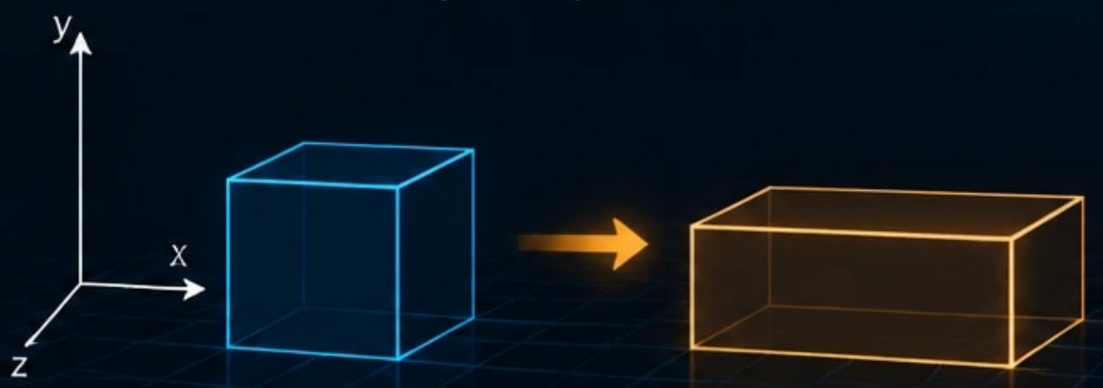


Contoh: $s_x = 2.0,\ s_y = 2.0$

NON-UNIFORM SCALING

$$s_x \neq s_y$$

Proporsi object berubah.



Contoh: $s_x = 2.0,\ s_y = 0.5$

Inti Konsep

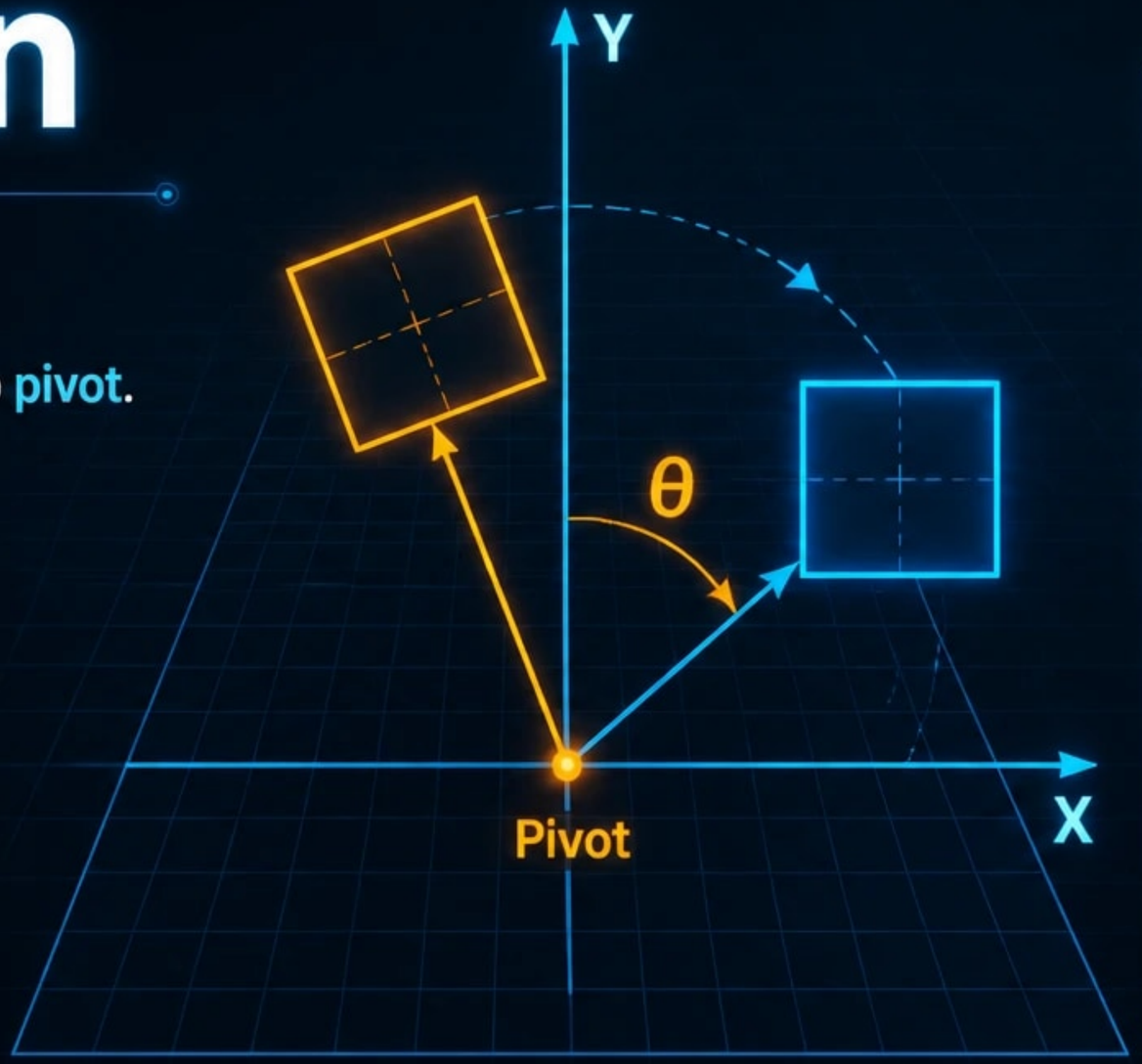
Uniform Scaling mempertahankan proporsi object.
Non-Uniform Scaling mengubah proporsi object.

Rotation

Rotation mengubah **orientasi** object terhadap **pivot**.

Diperlukan: **angle θ**

Dalam 2D biasanya rotasi terjadi pada bidang **XY**.



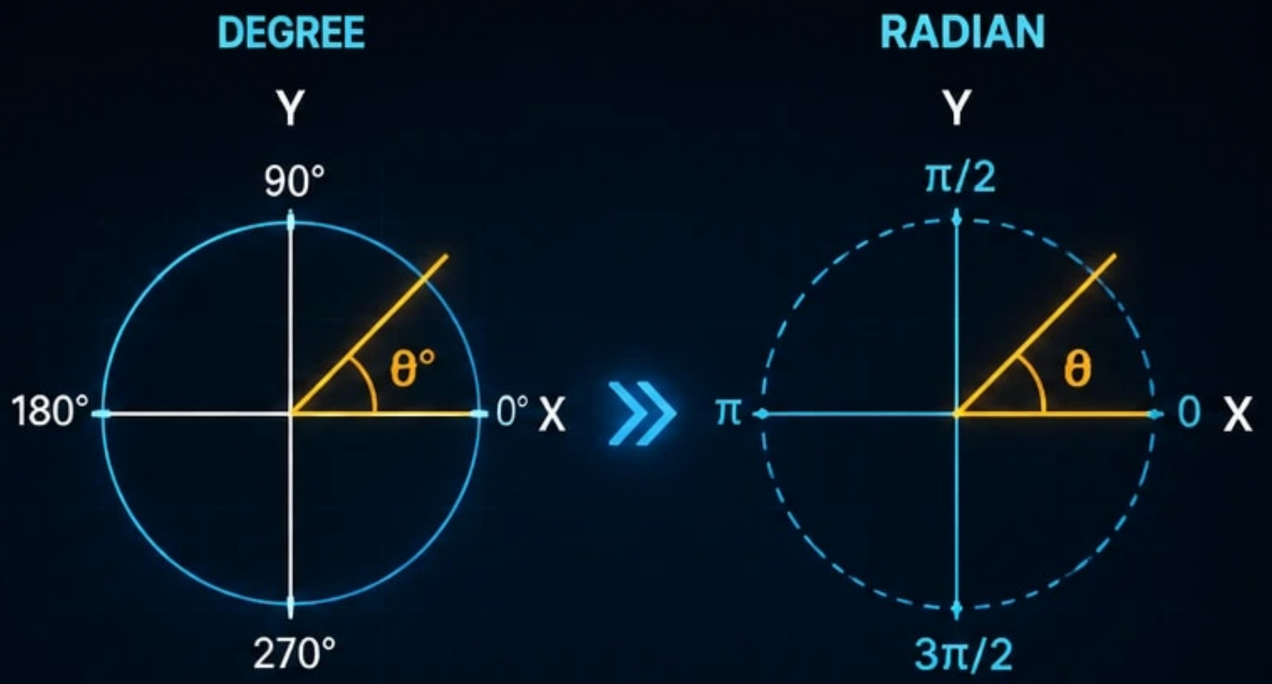
Rotasi 2D pada bidang **XY**

Sudut rotasi: angle **θ**

θ

Arah positif: Counter-Clockwise

Degree dan Radian



Sudut dapat dinyatakan dalam:

- degree
- rad | radian

KONVERSI

$$\text{radian} = \text{degree} \times \frac{\pi}{180}$$

Fungsi trigonometri pada pemrograman umumnya menggunakan **radian**.

Rumus Rotation 2D

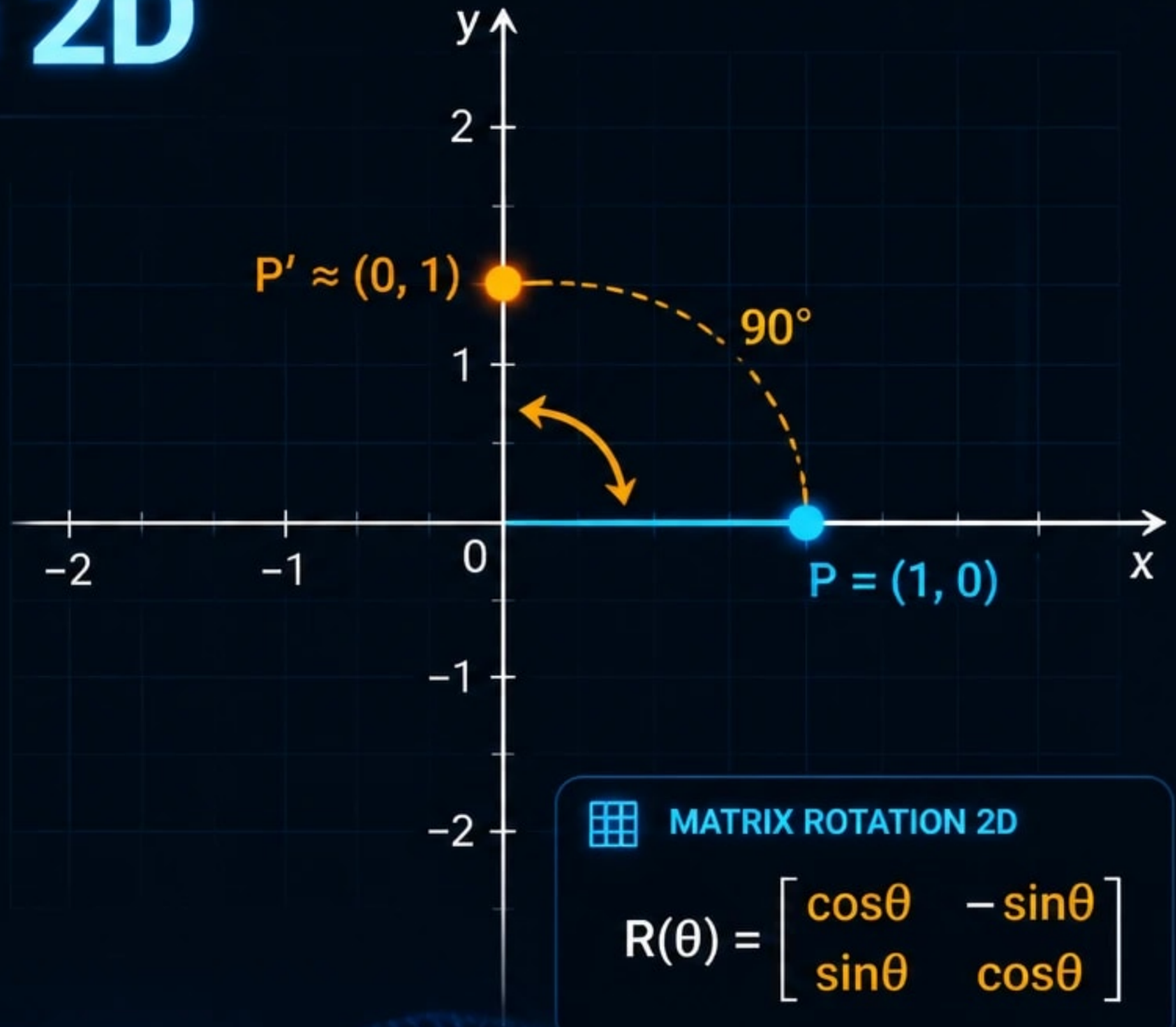
RUMUS ROTATION 2D

$$x' = x\cos\theta - y\sin\theta$$

$$y' = x\sin\theta + y\cos\theta$$

CONTOH

- P = (1, 0)
- Rotation = 90°
- → P′ ≈ (0, 1)



MATRIX ROTATION 2D

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Masalah Jika Tanpa Matrix

Translation, scaling, rotation dapat dihitung satu per satu.

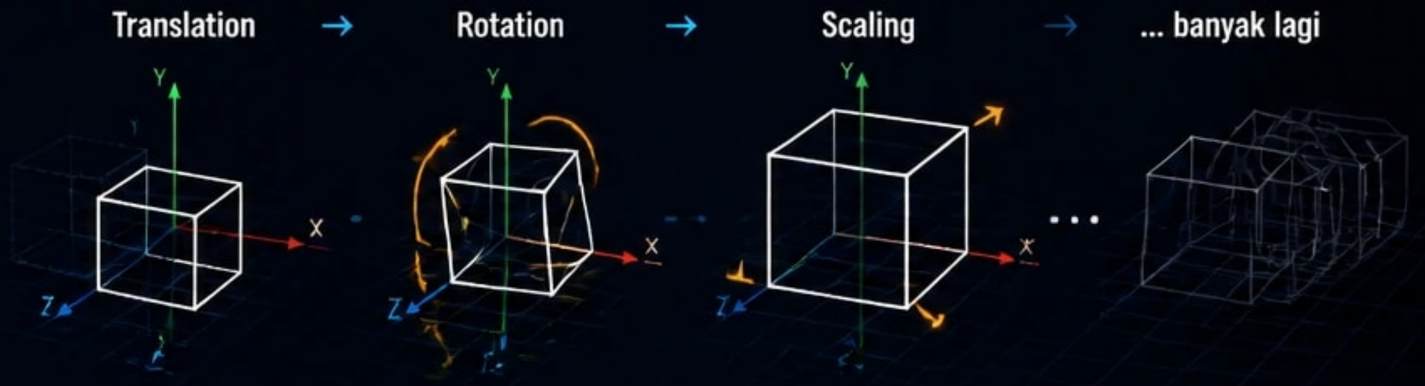
Namun ketika transformasi banyak:

- rumus menjadi **sulit dikelola**,
- urutan **sulit dilacak**,
- sulit dikirim secara **efisien** ke GPU.

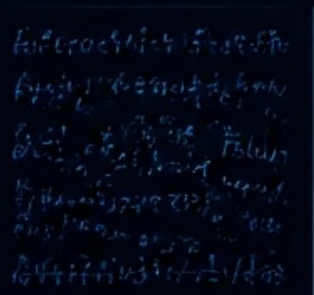
Solusi:

MATRIX

Tanpa Matrix: Transformasi Bertumpuk



Akibatnya:



✕ Rumus menjadi **sulit** dikelola



✕ Urutan sulit **dilacak**

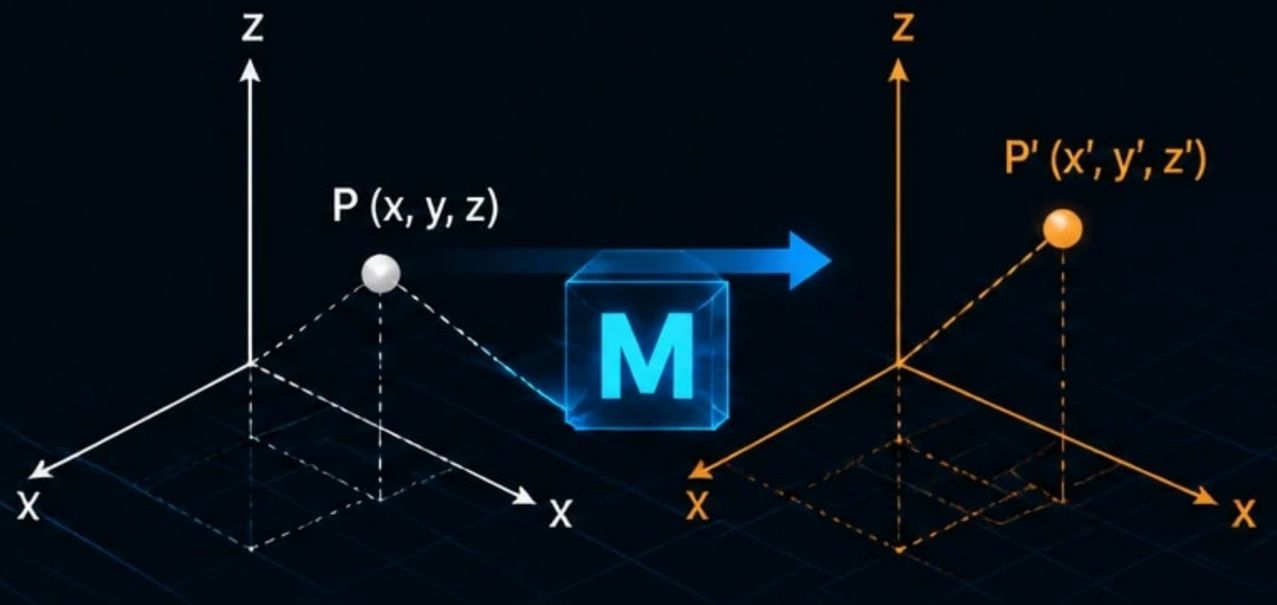


✕ Sulit dikirim secara **efisien** ke GPU

Solusi:

$$\begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ m_{30} & m_{31} & m_{32} & m_{33} \end{bmatrix}$$

Apa Itu Matrix?



Matrix adalah susunan angka dalam baris dan kolom.

Contoh:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Dalam grafika:

$$P' = M \times P$$

Matrix digunakan untuk mentransformasi vector.

Mentransformasi Vector

Vector Asal **P**

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$\times$

Matrix Transformasi

$$\begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \\ m_{41} & m_{42} & m_{43} & m_{44} \end{bmatrix}$$

$=$

Vector Hasil **P'**

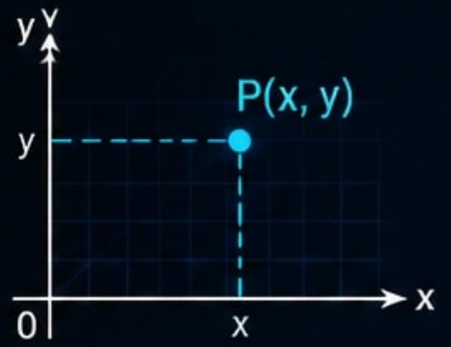
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix}$$

Vector dan Matrix



Titik 2D dapat ditulis:

$$P = \begin{bmatrix} x \\ y \end{bmatrix}$$



Transformasi linear:

$$P' = M \times P$$

Scaling dan rotation dapat direpresentasikan langsung dengan matrix 2×2.

SCALING

$$S = \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix}$$



$$P' = \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix} \times \begin{bmatrix} x \\ y \end{bmatrix}$$

ROTATION (θ)

$$R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$



$$P' = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \times \begin{bmatrix} x \\ y \end{bmatrix}$$

Scaling Matrix 2×2

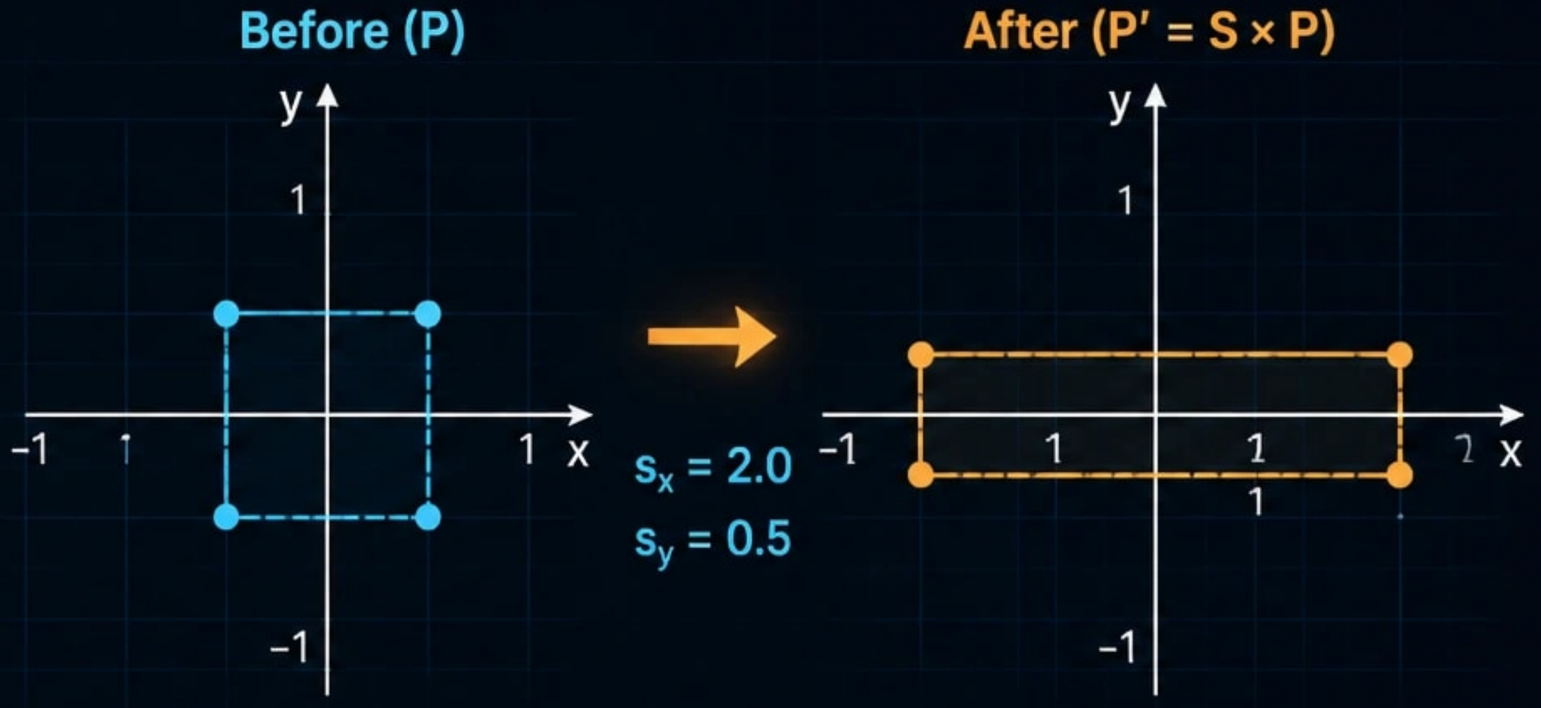
Scaling Matrix 2×2

$$S = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

Maka:

$$P' = S \times P$$

Matrix membuat transformasi lebih **konsisten.**

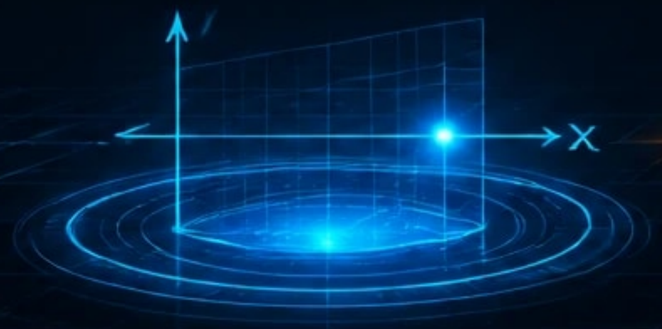


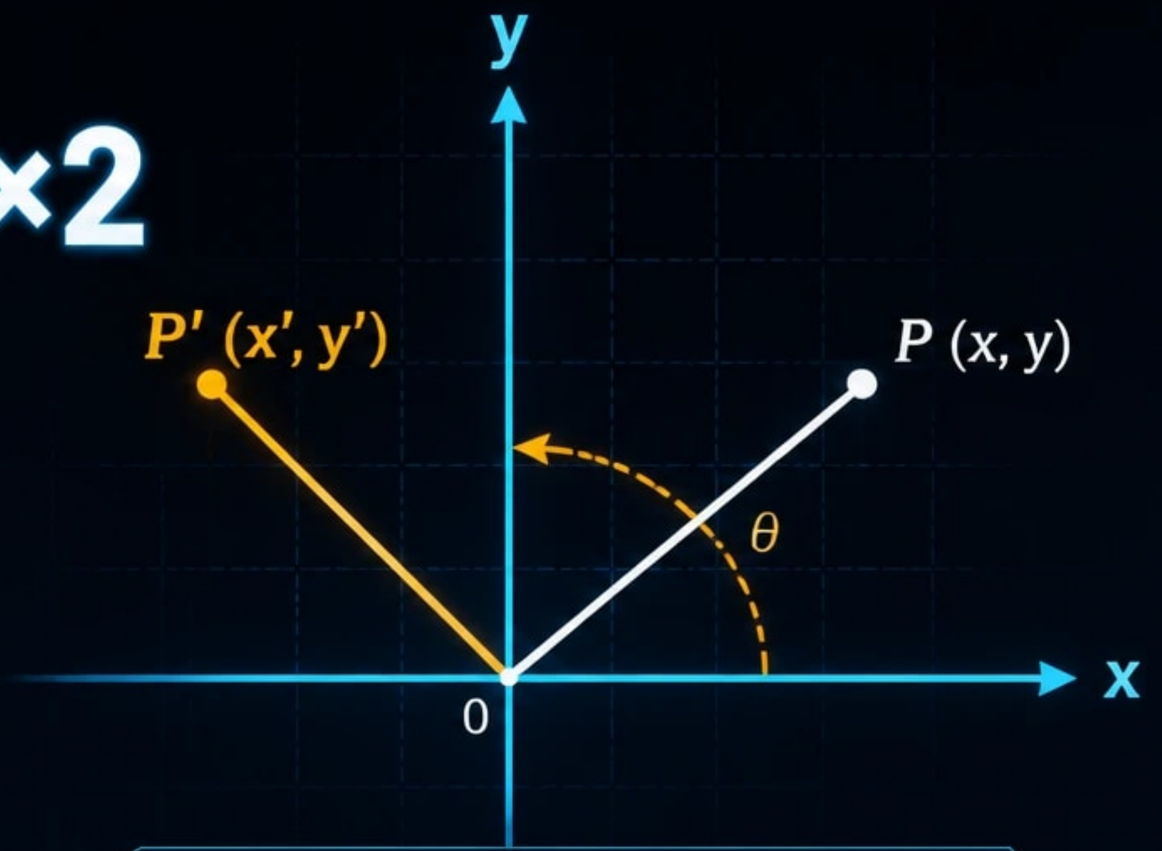
$$P = \begin{bmatrix} x \\ y \end{bmatrix} \blacktriangleright P' = \begin{bmatrix} x' \\ y' \end{bmatrix} \blacktriangleright \begin{aligned} x' &= s_x \cdot x \\ y' &= s_y \cdot y \end{aligned}$$

Rotation Matrix 2×2

$$R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

$$P' = R \times P$$





- Rotasi terhadap origin (0,0)
- Berlawanan arah jarum jam ($\theta > 0$)
- Mempertahankan jarak & sudut

Mengapa Translation Berbeda?



Translation

menggunakan penjumlahan:

$$x' = x + t_x$$
$$y' = y + t_y$$

Tidak dapat direpresentasikan langsung dengan **matrix linear 2×2**.

Solusi:

Homogeneous Coordinate



Mengapa matrix 2×2 tidak cukup?

Matrix linear 2×2 hanya dapat:

- Rotasi
- Skalasi
- Shearing

Matrix 2×2

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

✗ Tidak dapat menambah offset (t_x, t_y)

Dengan Homogeneous Coordinate (3×3)

Translation matrix:

$$T(t_x, t_y) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

✓ Dapat merepresentasikan translation sebagai operasi linear

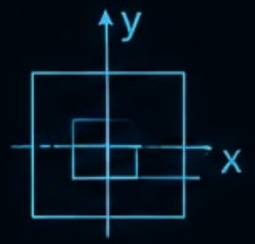
Homogeneous coordinates (koordinat homogen) adalah sistem koordinat yang digunakan dalam geometri projektif dan grafika komputer untuk merepresentasikan titik-titik di ruang spasial.

Jika dalam sistem koordinat Kartesius sebuah titik di ruang 2D dinyatakan dengan dua angka (x, y), maka dalam koordinat homogen titik tersebut dinyatakan dengan tiga angka (x, y, w), di mana [w] adalah faktor skala non-nol.

Mengapa Koordinat Homogen Dibutuhkan?
* Dalam grafika komputer (seperti OpenGL, WebGL, atau game engine), objek perlu digeser (translasi), diputar (rotasi), dan diubah ukurannya (skala).
* Rotasi dan skala dapat direpresentasikan dengan mudah menggunakan perkalian matriks (2 x 2) (untuk 2D).
* Namun, translasi (pergeseran) membutuhkan operasi penjumlahan vektor.
* Dengan koordinat Kartesius biasa, kita tidak bisa menyatukan rotasi, skala, dan translasi ke dalam satu buah matriks terpadu.

Koordinat homogen memecahkan masalah ini dengan menambah satu dimensi ekstra [w], sehingga semua transformasi geometri (termasuk translasi) dapat dilakukan hanya dengan satu perkalian matriks standar (dikenal sebagai Affine Transformation).

Homogeneous Coordinate



2D POINT

Titik 2D

(x, y)

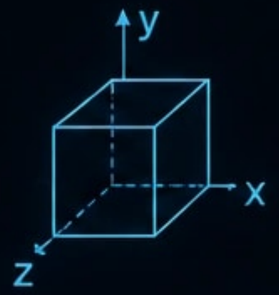
diubah menjadi

(x, y, 1)

Sehingga dapat memakai matrix 3×3.

MATRIX 3×3

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$



3D POINT

Titik 3D

(x, y, z)

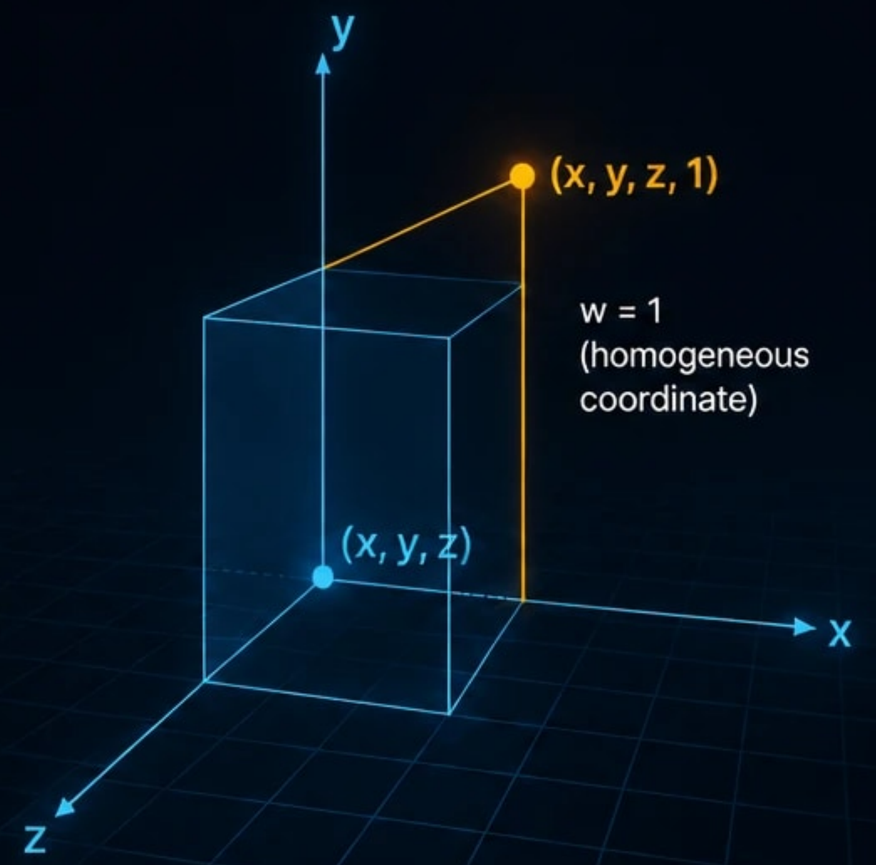
diubah menjadi

(x, y, z, 1)

menggunakan matrix 4×4.

MATRIX 4×4

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}$$



Dengan menambahkan komponen w = 1,
kita dapat menyatukan translasi, rotasi, skala, dan proyeksi dalam **operasi matrix**.

TRANSLATION MATRIX 2D

MATRIX TRANSLATION 2D

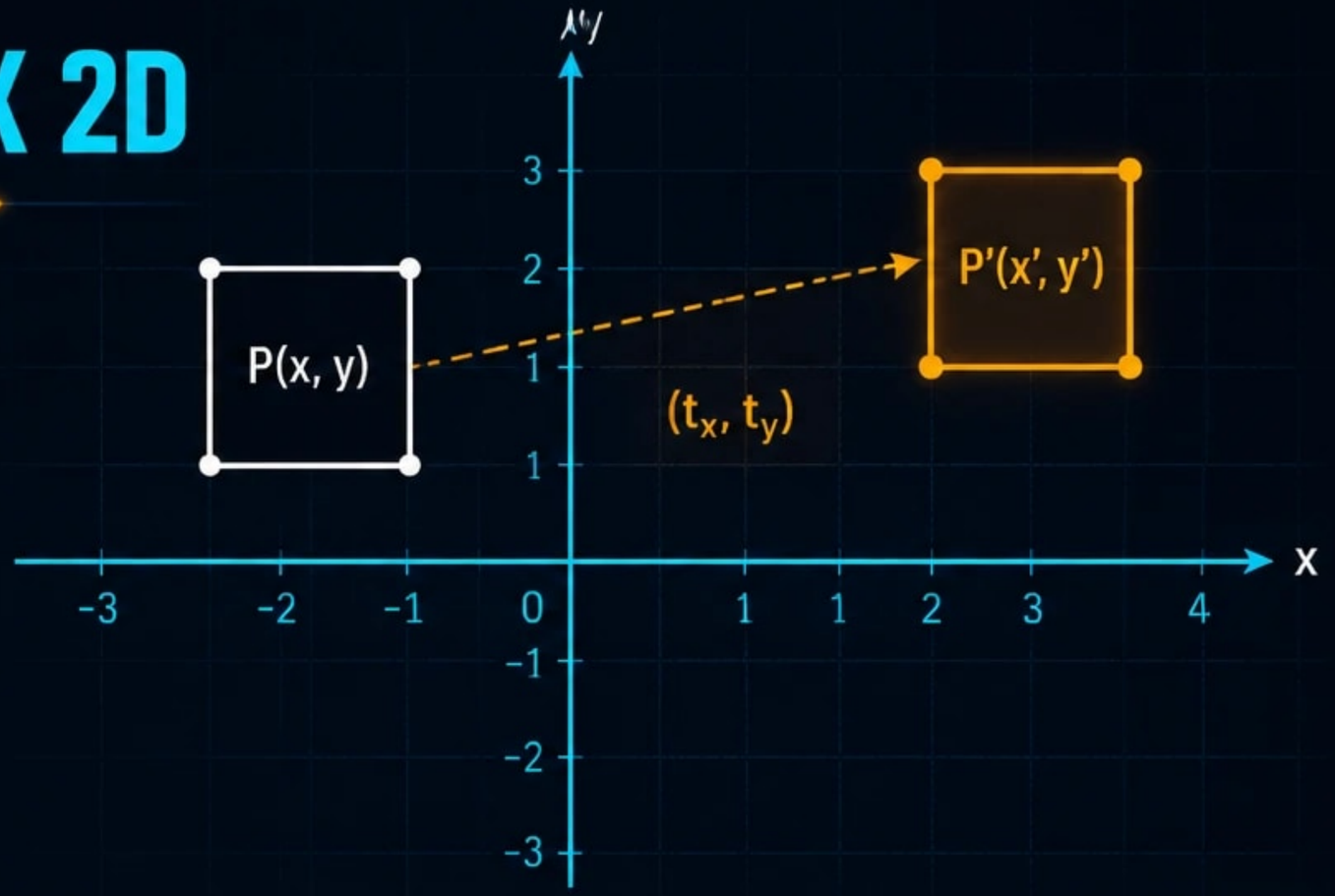
$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

HOMOGENEOUS COORDINATE

$$P' = T \times P$$

KEY POINT

Translation kini dapat digabung dengan transformasi lain.



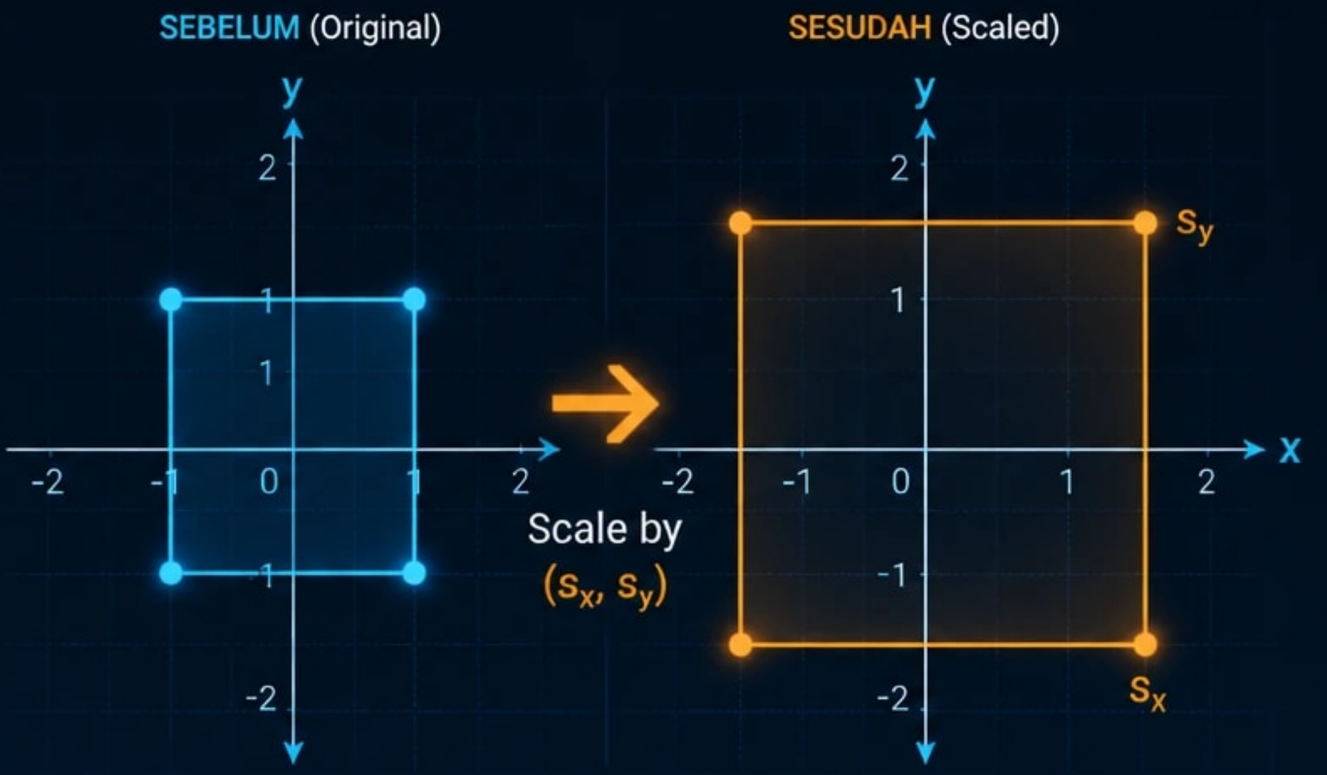
t_x menggeser objek pada sumbu X (kanan/kiri)

t_y menggeser objek pada sumbu Y (atas/bawah)

Scaling Matrix 3×3

$$S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Format 3×3 membuat scaling kompatibel dengan translation 2D.



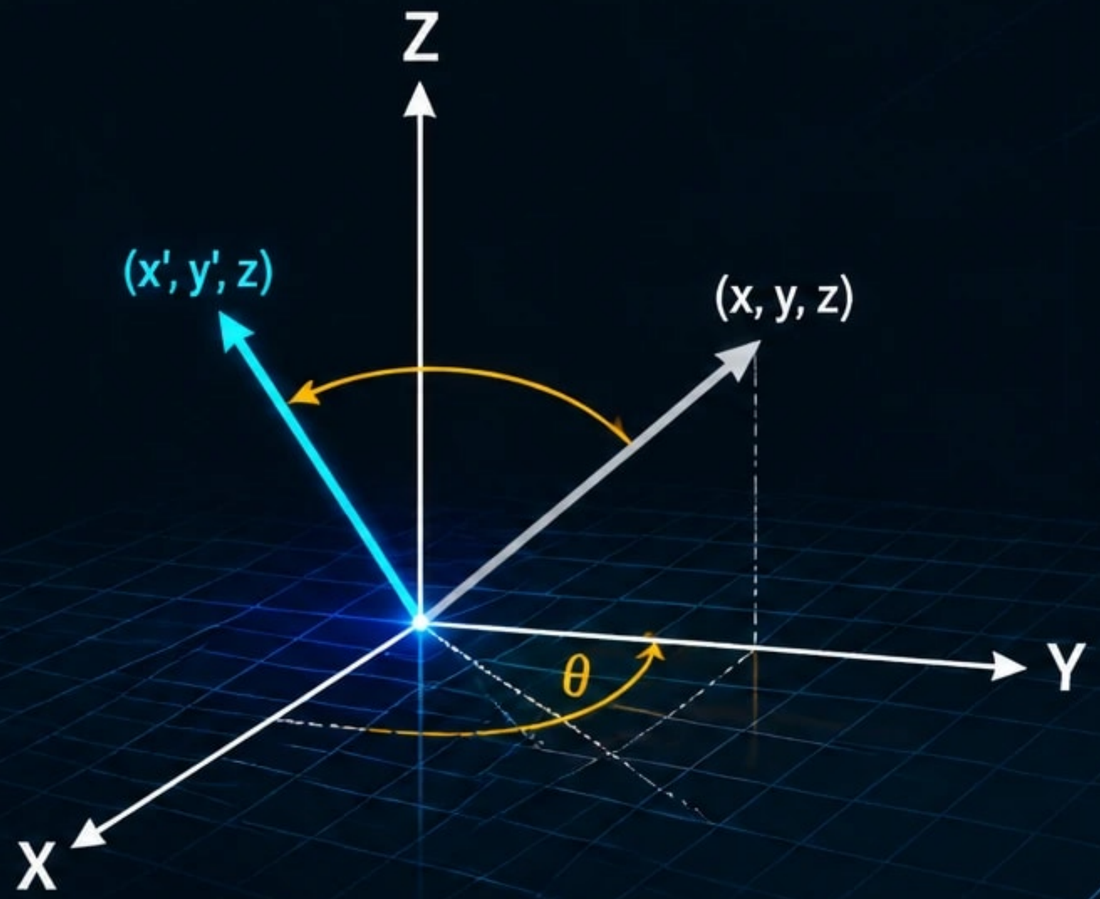
$$(x, y, 1) \longrightarrow (s_x x, s_y y, 1)$$

Rotation Matrix 3×3

$$R = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



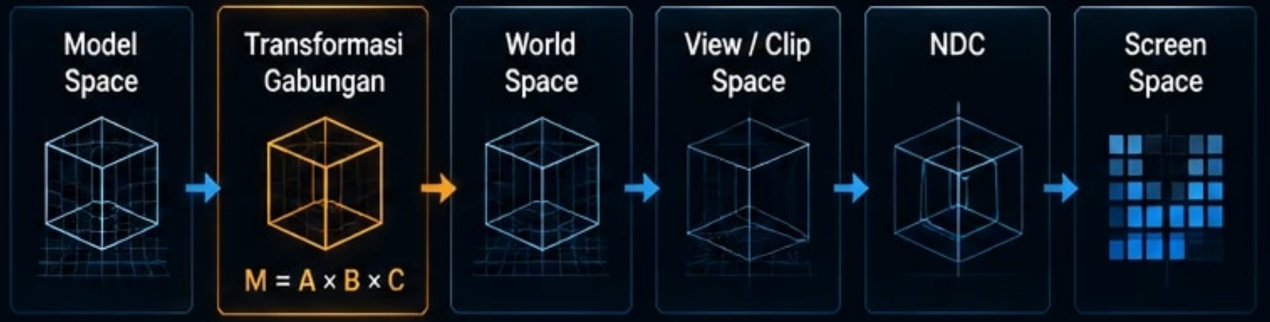
Sekarang semua transformasi dasar memiliki bentuk **matrix yang seragam.**





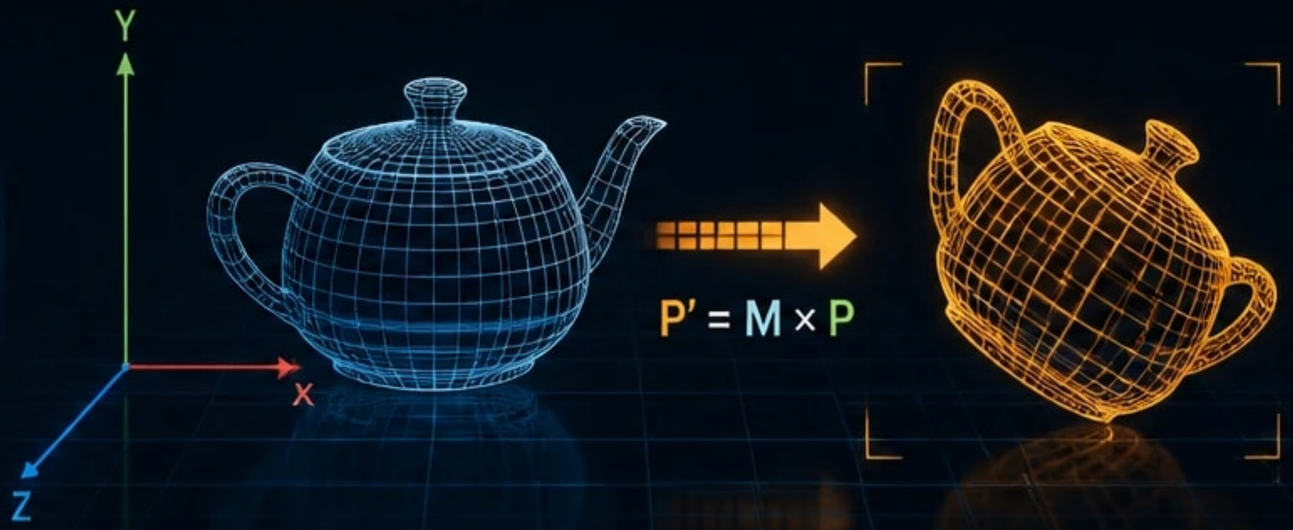
Grafika Komputer

Matrix Multiplication



Transformasi dapat digabung:

$$M = \overset{A}{\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}} \times \overset{B}{\begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{bmatrix}} \times \overset{C}{\begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & c_{34} \\ c_{41} & c_{42} & c_{43} & c_{44} \end{bmatrix}} \overset{C}{}$$



Kemudian satu matrix digunakan untuk seluruh vertex:

$$P' = M \times P$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ w' \end{bmatrix} = \underset{4\times4}{M} \times \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}$$

Ini lebih efisien daripada menghitung setiap transformasi secara terpisah.

- Gabungkan transformasi (translate, rotate, scale, dll.) menjadi satu matrix.
- Kalikan satu matrix dengan setiap vertex.
- Lebih sedikit operasi, lebih cepat di CPU/GPU.

URUTAN TRANSFORMASI PENTING

Matrix multiplication: tidak komutatif.

$$A \times B \neq B \times A$$

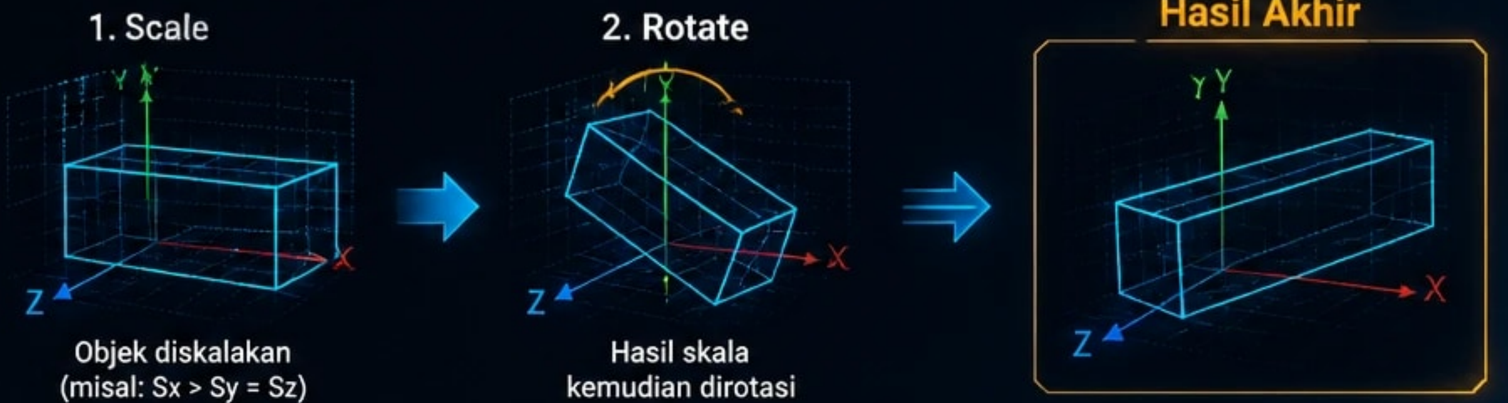
Artinya:

Scale → Rotate

dapat memberi hasil berbeda dari:

Rotate → Scale

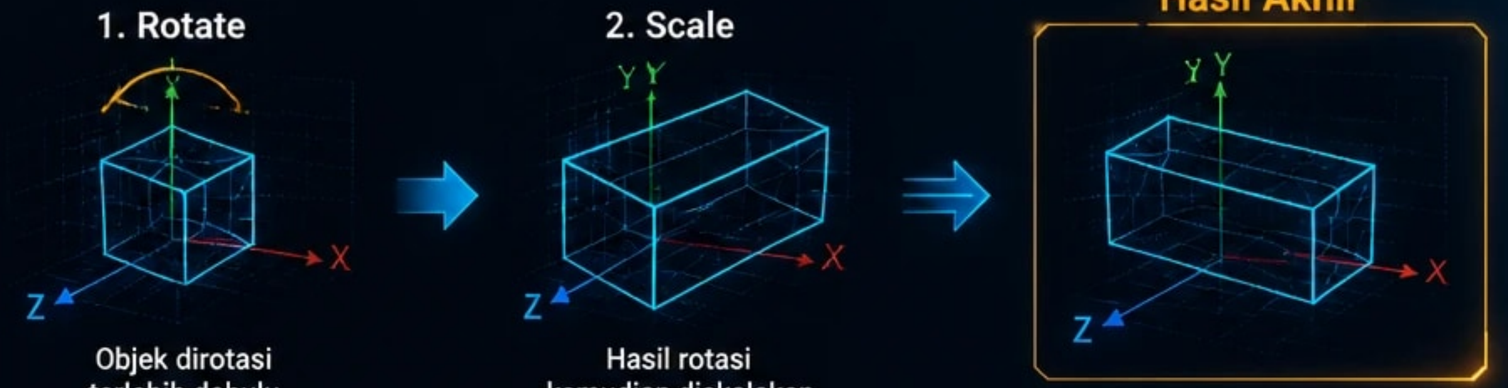
1. Scale → 2. Rotate



Objek diskalakan (misal: Sx > Sy = Sz)

Hasil skala kemudian dirotasi

1. Rotate → 2. Scale



Objek dirotasi terlebih dahulu

Hasil rotasi kemudian diskalakan

Urutan transformasi berpengaruh pada hasil akhir. Pilih urutan yang sesuai dengan tujuan visual Anda.

Grafika Komputer

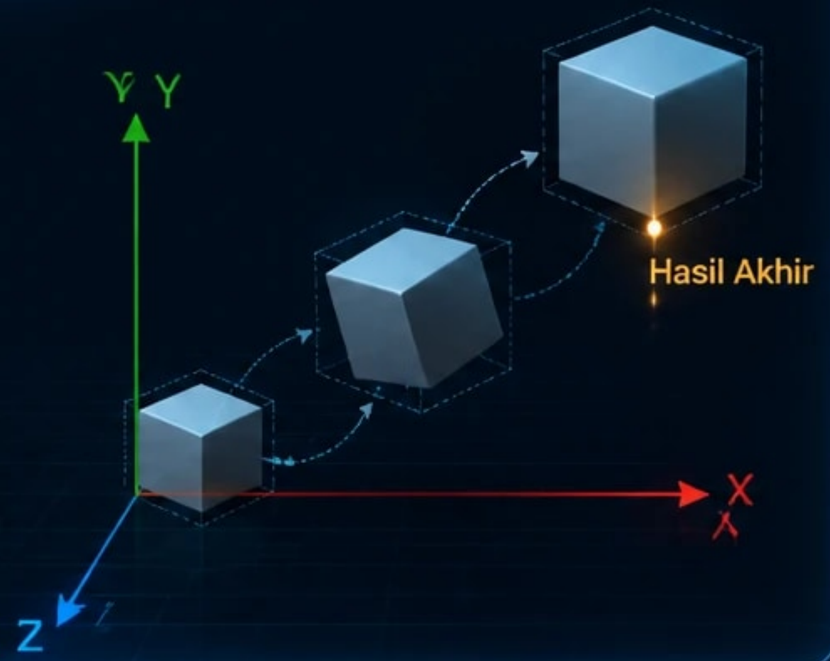
Contoh Transform Order

KASUS A

Scale

Rotate

Translate

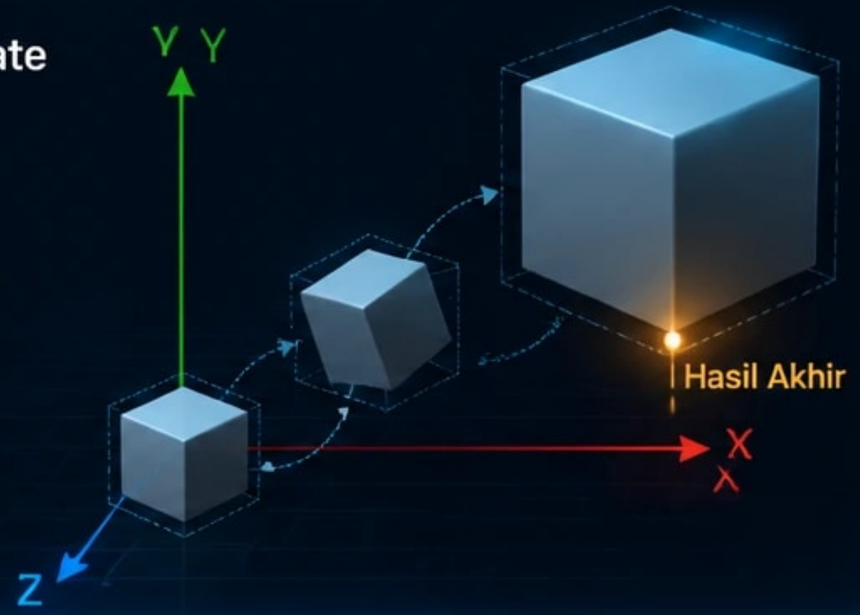


KASUS B

Translate

Rotate

Scale



Hasil posisi dan orientasi object dapat **sangat berbeda.**



Grafika Komputer

MODEL MATRIX

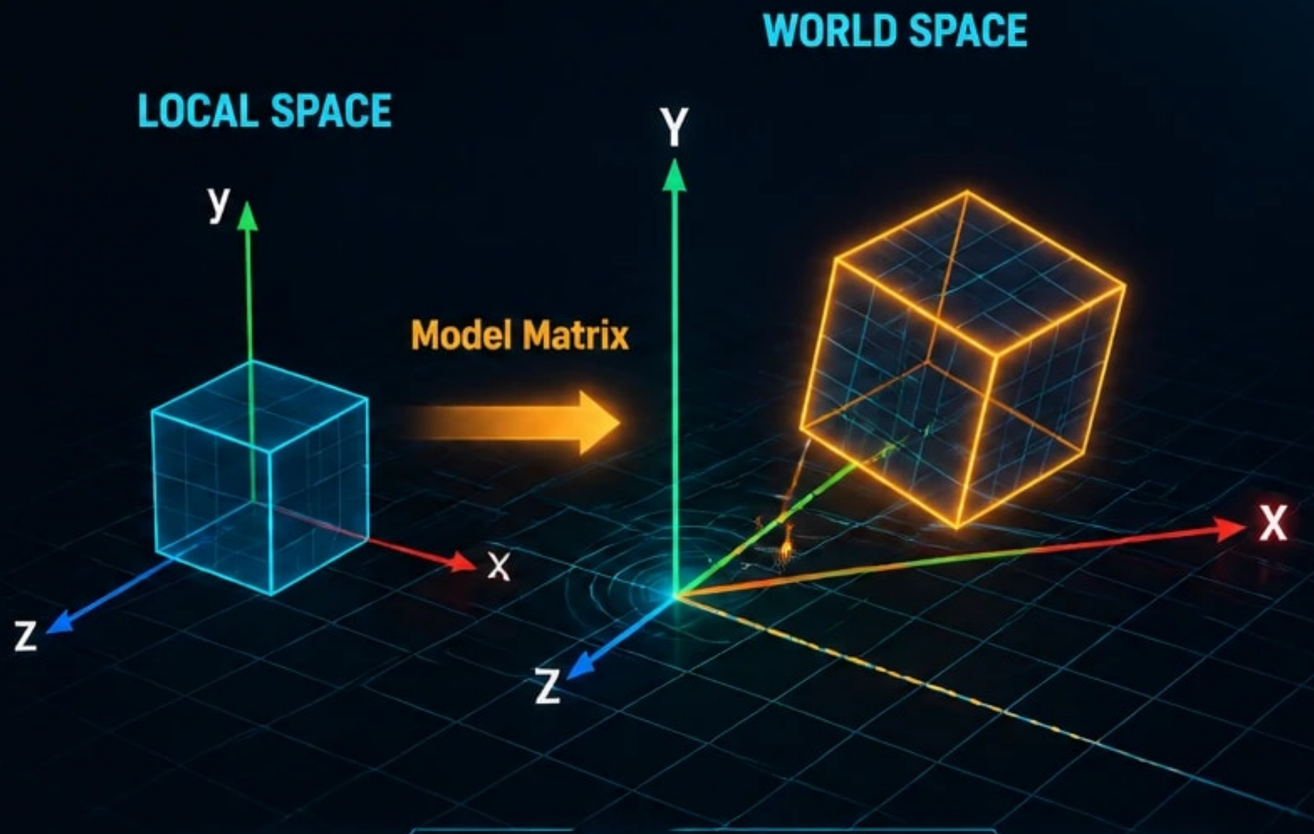
Model Matrix mengubah:



Secara konseptual:

worldPosition =
ModelMatrix × localPosition

- Memposisikan, mengrotasi, dan menskalakan objek
- Dari koordinat lokal objek ke koordinat dunia
- Representasi dalam bentuk matriks 4×4





Grafika Komputer

Transformasi 3D Menggunakan Matrix 4×4

$$\begin{bmatrix} x' \\ y' \\ z' \\ w' \end{bmatrix} = M_{4\times4} \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}$$



Dalam 3D digunakan **matrix 4×4.**

Representasi umum:

$$\begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}$$

x, y, z → posisi/arah di 3D

w → komponen homogen (w=1 untuk posisi)

Transformasi 3D digabungkan dalam satu **matrix 4×4** melalui perkalian matrix.

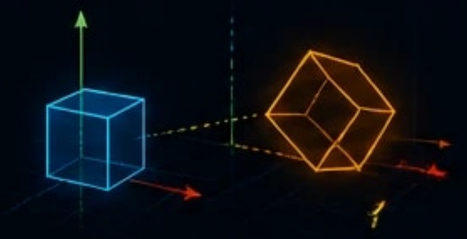
Model Matrix 4×4 dapat memuat:

Translation (Pergeseran)



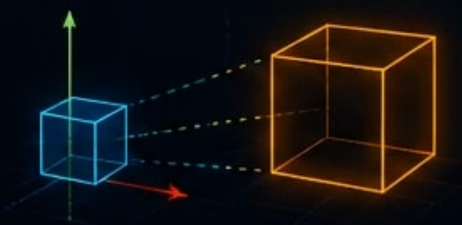
$$\begin{bmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation (Rotasi)



$$\begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Scaling (Skalasi)



$$\begin{bmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Translation Matrix 3D

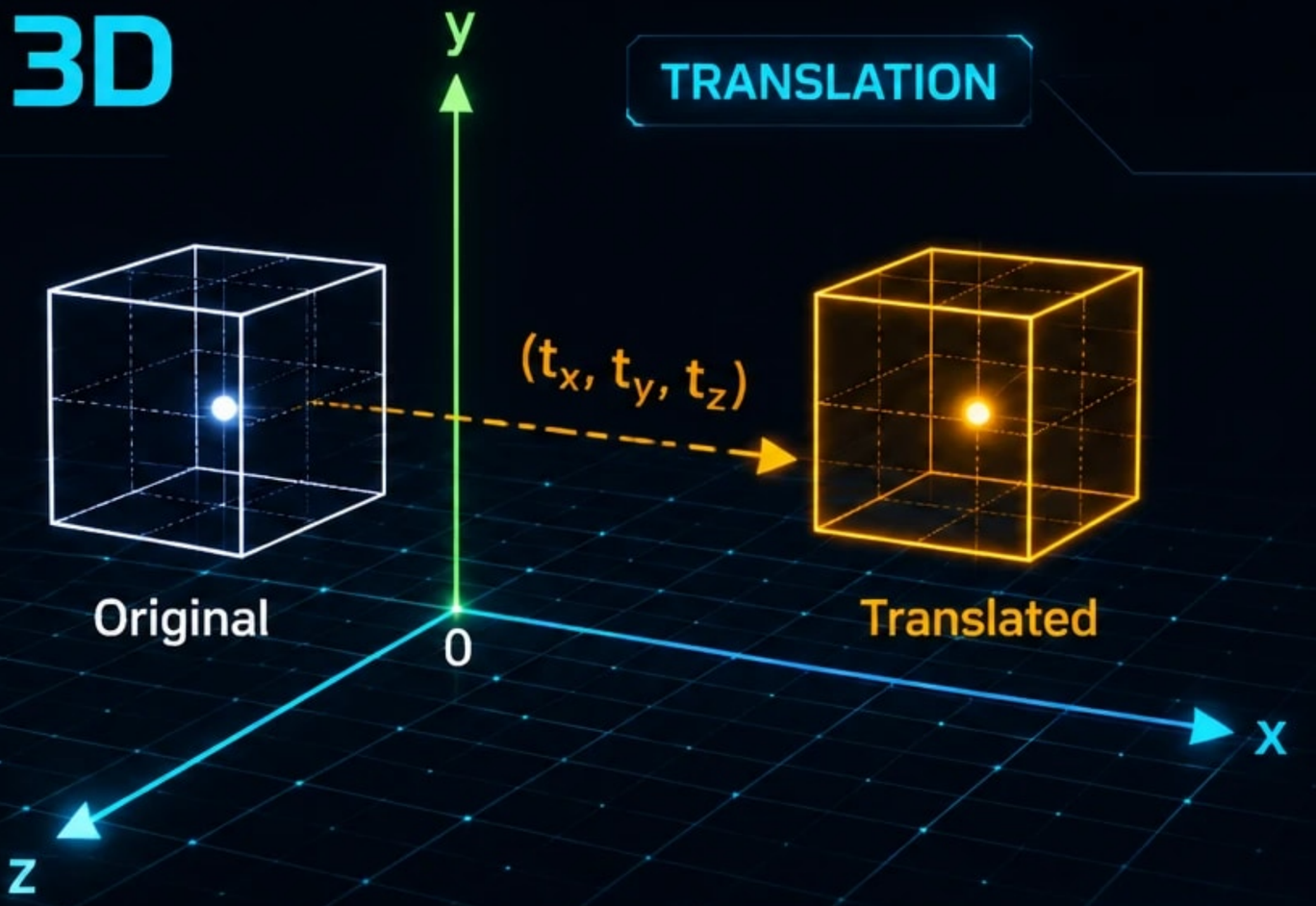
TRANSLATION

MATRIX TRANSLASI 3D

$$T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



t_x, t_y, t_z menentukan perpindahan object di world space.



- t_x → geser sepanjang sumbu X
- t_y → geser sepanjang sumbu Y
- t_z → geser sepanjang sumbu Z

Contoh

$t_x = 2.0$, $t_y = 1.0$, $t_z = -1.5$

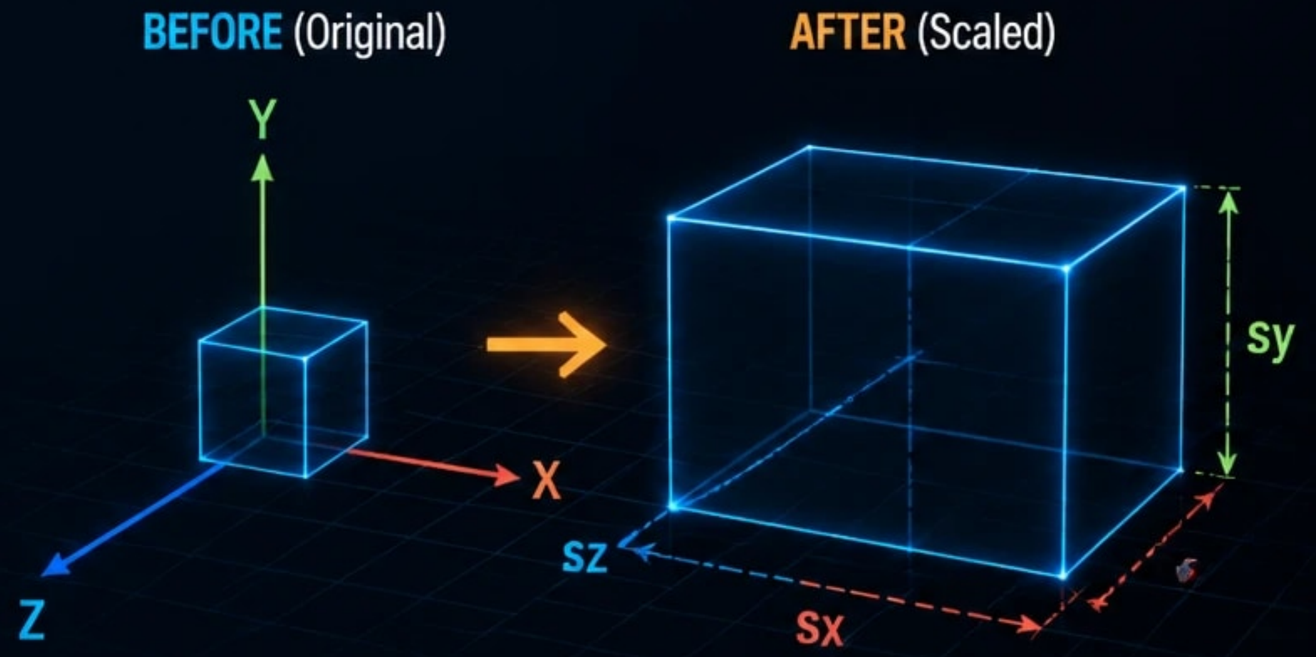
Object bergeser menuju arah +X, +Y, dan -Z

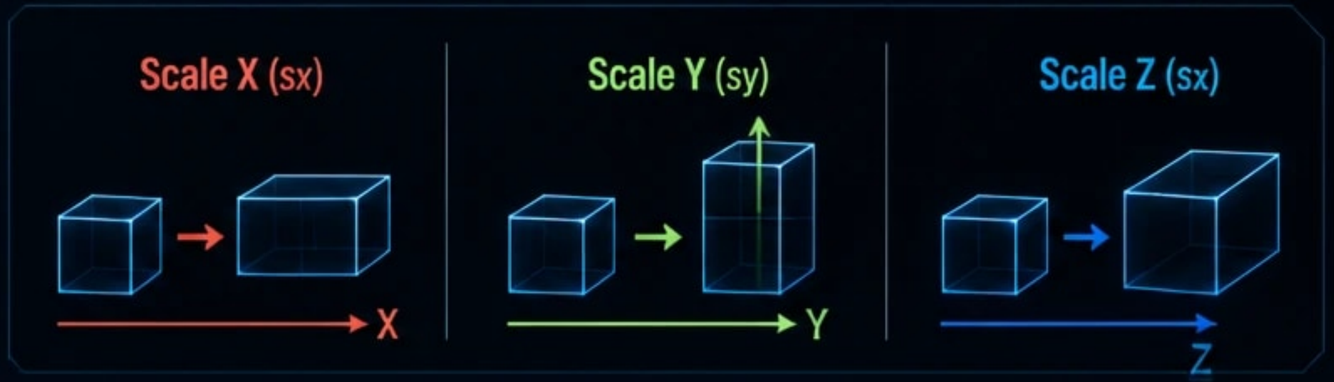
Scaling Matrix 3D

$$S = \begin{bmatrix} Sx & 0 & 0 & 0 \\ 0 & Sy & 0 & 0 \\ 0 & 0 & Sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Scaling dapat berbeda untuk sumbu X, Y, Z.





Rotation dalam 3D

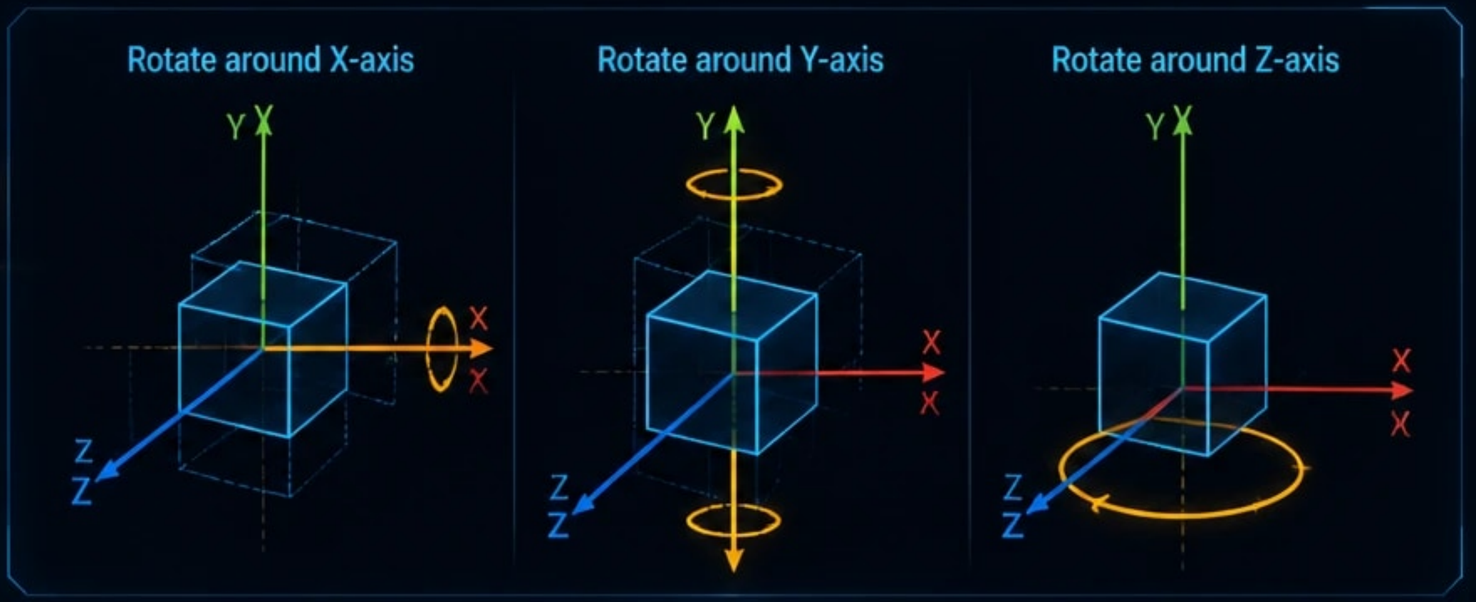
Rotation 3D dapat terjadi terhadap:

- X-axis
- Y-axis
- Z-axis

Setiap axis memiliki **rotation matrix** sendiri.

Pada praktikum fokus dapat dimulai dari:

- Rotation Z untuk 2D
- Rotation Y untuk object 3D sederhana



Rotation Matrices

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Uniform Matrix pada WebGL

Dari Pertemuan 2 kita sudah mengenal **uniform**.

Sekarang digunakan untuk mengirim **matrix**:

```
uniform mat4 u_model;
```

Uniform:

- sama untuk semua vertex
- dalam satu **draw call**



Vertex Shader (GLSL)

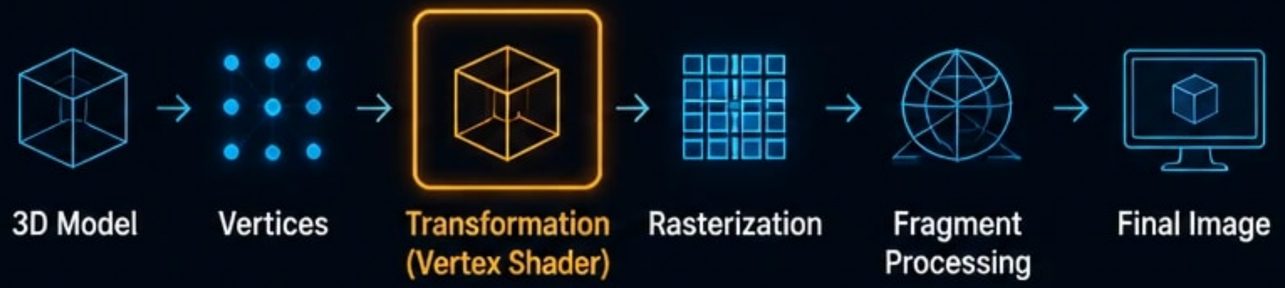
```
uniform mat4 u_model;
uniform mat4 u_view;
uniform mat4 u_projection;

in vec3 a_position;

void main() {
 gl_Position =
  u_projection * u_view *
 u_model * vec4(a_position, 1.0);
}
```



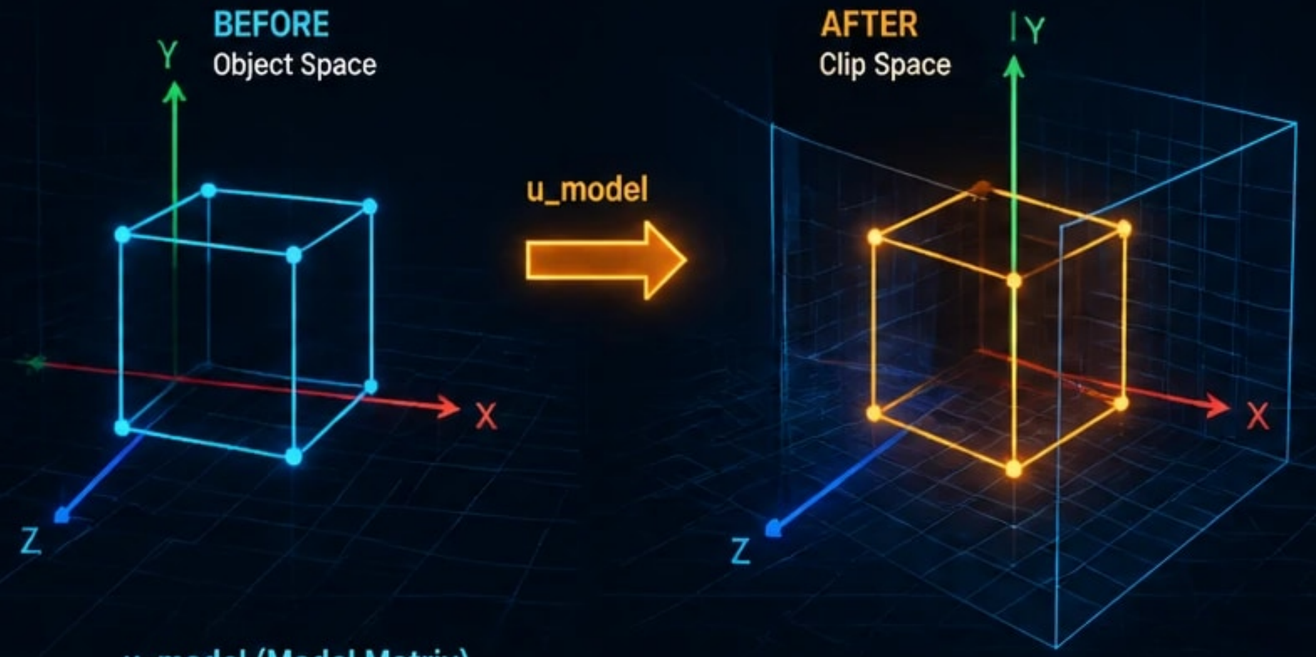
Transformation di Vertex Shader



GLSL

```
in vec3 a_position;
uniform mat4 u_model;

void main() {
   gl_Position =
      u_model *
       vec4(a_position, 1.0);
}
```



Catatan:
Untuk tahap ini, matrix lain dapat dibuat identity.



u_model (Model Matrix)

$$\begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix} = \text{Identity } I_4$$

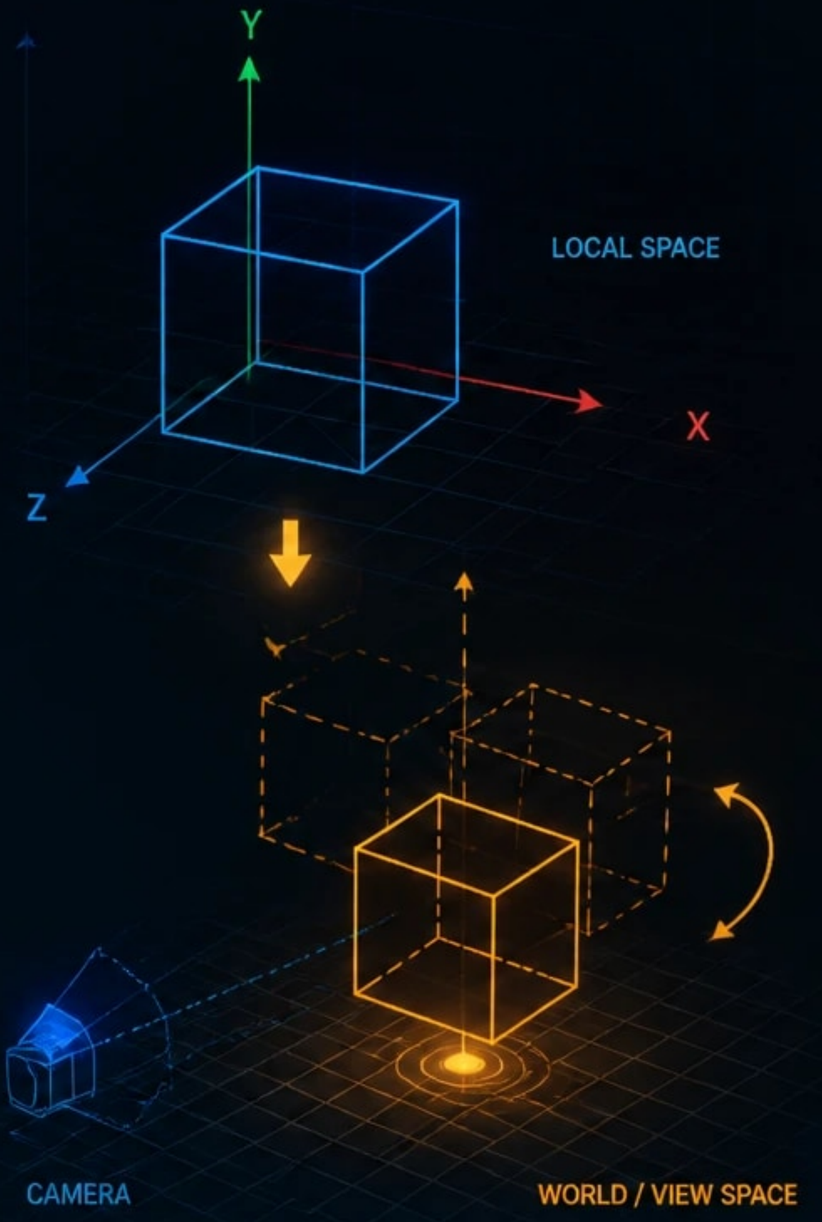
Matrix lain (view, projection) dibuat identity untuk tahap ini.

Geometry Tetap, Transform Berubah

Keuntungan:

- tidak perlu menulis ulang seluruh vertex buffer hanya untuk memindahkan object.

Vertex Buffer	Data posisi vertex disimpan di GPU (tetap)
Geometry Tetap	Topologi & posisi lokal objek tidak berubah
Model Matrix	Berubah per frame (translate, rotate, scale)
Vertex Shader	Menerapkan transformasi ke setiap vertex
Transformed Vertex	Vertex siap untuk rasterization

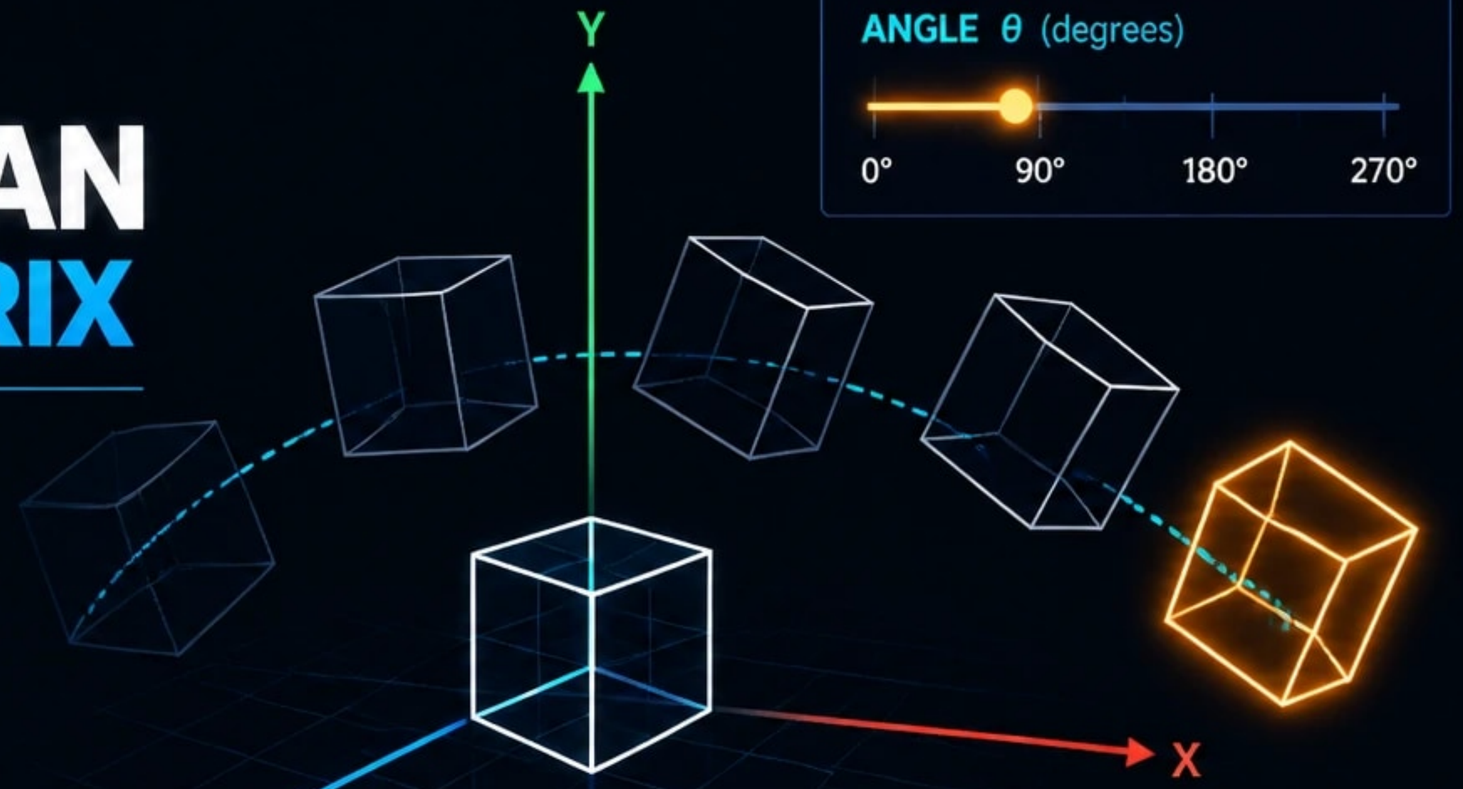


Grafika Komputer

ANIMASI DENGAN TRANSFORM MATRIX

1. Time
2. Update Angle
3. Build Rotation Matrix
4. Send Uniform
5. Draw

Ini adalah dasar animasi transformasi real-time.



ANGLE θ (degrees)

0° 90° 180° 270°

ROTATION MATRIX (Y-AXIS)

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

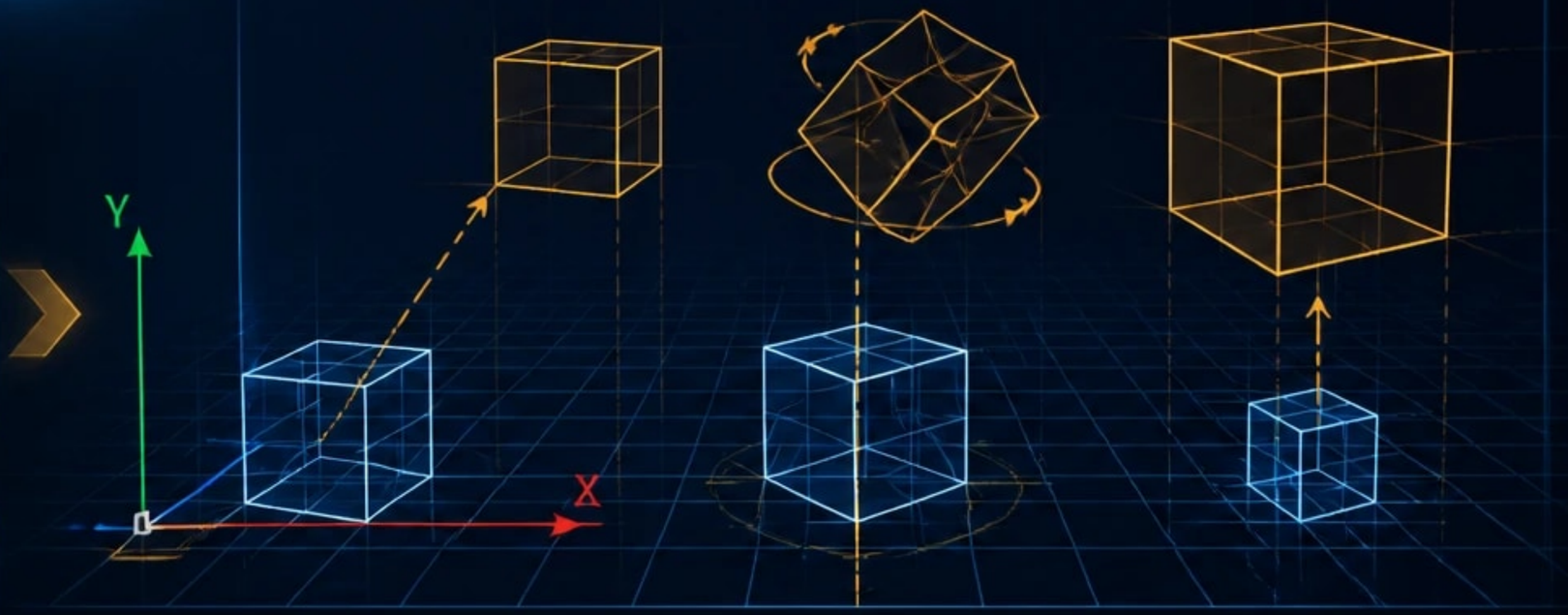
TIME t

Interaksi User

TRANSLATION ROTATION SCALING

Input dapat mengubah:

- translation
- rotation
- scaling



Contoh Kontrol

↑ ↓ ← →	→	Translate
A / D	→	Rotate
+ / -	→	Scale

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}$$

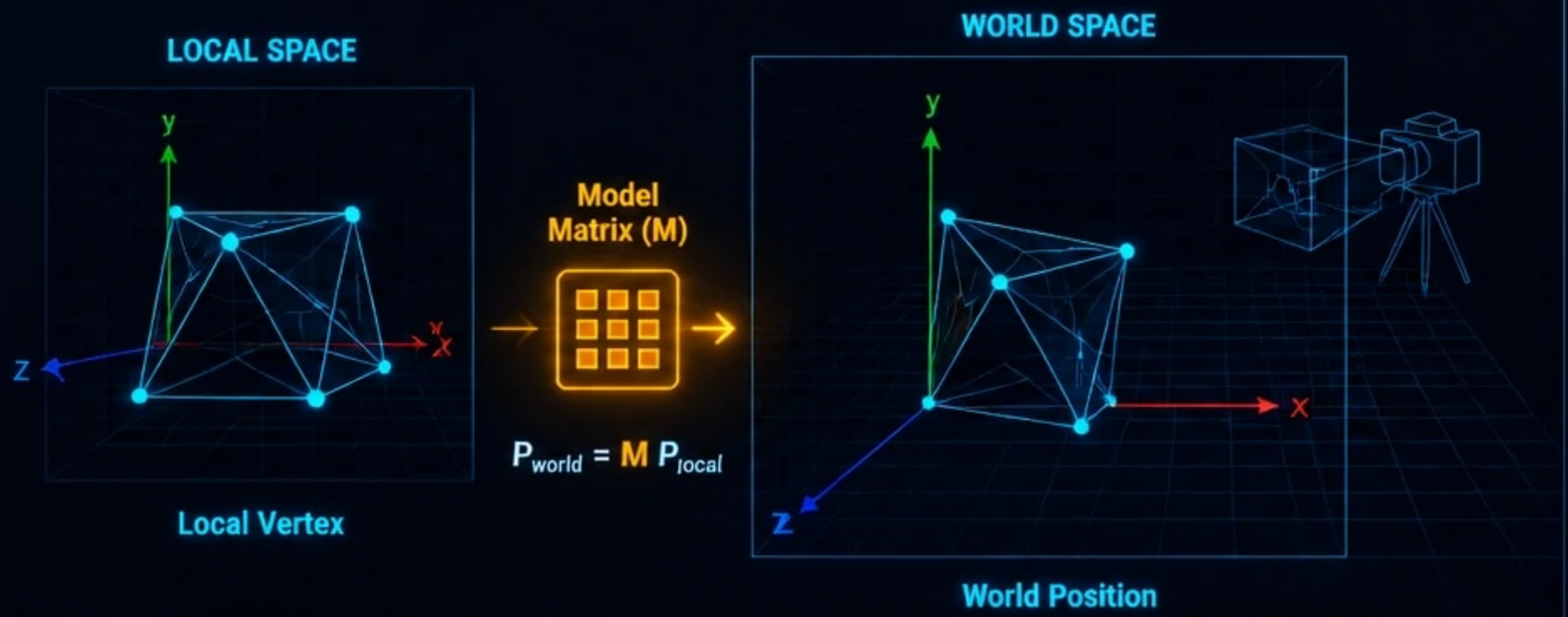
Transformasi diproses melalui **matrix.**

Pipeline Pertemuan 3

- Local Vertex
- Model Matrix
- World Position
- Vertex Shader Output
- Rasterization
- Fragment Shader

FOKUS P3:

LOCAL → WORLD





Grafika Komputer

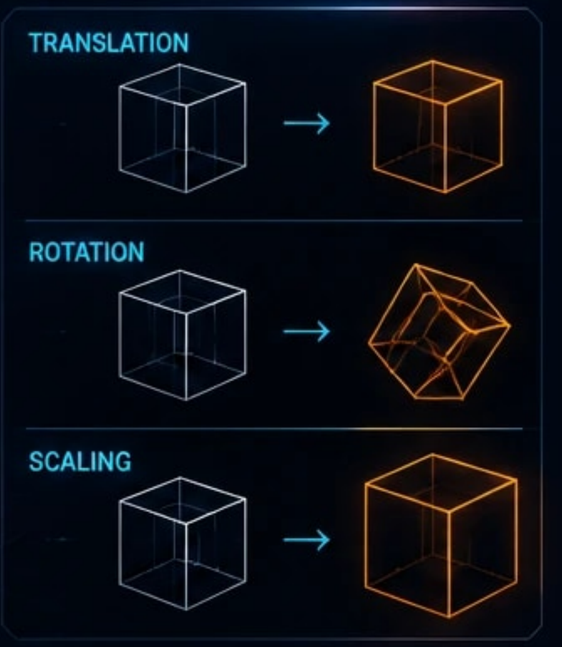
Praktikum:
Interactive Transformation



Mahasiswa mengembangkan program WebGL menjadi object yang dapat:

- translation
- rotation
- scaling
- menggabungkan beberapa transformasi
- berubah melalui input pengguna
- dianimasikan dengan `requestAnimationFrame()`

Fokus:
matrix-based transformation.









TERIMA KASIH

Pertemuan Berikutnya

Camera, Projection & 3D

